

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO
ĐỒ ÁN CHUYÊN NGÀNH

TIẾP CẬN HỌC LỒNG NHAU
TRONG MÔ HÌNH HỌC LIÊN TỤC
VÀ MỘT SỐ ỨNG DỤNG

NGÀNH: KHOA HỌC MÁY TÍNH

HỘI ĐỒNG: HỘI ĐỒNG 3L ĐỒ ÁN CHUYÊN NGÀNH

GVHD: PGS. TS. Lê Hồng Trang

GVHD: ThS. Dương Đức Tín

—o0o—

SVTH1: Bùi Trần Duy Khang (2311402)

SVTH2: Trần Anh Tài (2252726)

TP. HỒ CHÍ MINH, THÁNG 5 NĂM 2026



Lời cam đoan

Chúng em xin cam đoan Báo cáo Đồ án Chuyên ngành với đề tài "Tiếp cận học lồng nhau trong mô hình học liên tục và một số ứng dụng" là kết quả của quá trình tự nghiên cứu và thực hiện của tập thể nhóm sinh viên, dưới sự hướng dẫn của tập thể các giảng viên là PGS. TS. Lê Hồng Trang và ThS. Dương Đức Tín tại Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách khoa - ĐHQG TP.HCM

Các số liệu, hình ảnh, kết quả và nội dung trình bày trong báo cáo được thu thập, tổng hợp từ các nguồn tài liệu có uy tín, được trích dẫn rõ ràng theo đúng quy định; các phần tham khảo từ công trình của tác giả khác đều đã được ghi nguồn đầy đủ.

Nhóm sinh viên hoàn toàn chịu trách nhiệm trước Khoa Khoa học & Kỹ thuật Máy tính và Trường Đại học Bách khoa - ĐHQG TP.HCM về tính trung thực, nguyên gốc của toàn bộ nội dung báo cáo này.

TP. Hồ Chí Minh, tháng 5 năm 2026

Đại diện nhóm sinh viên thực hiện

Bùi Trần Duy Khang



Lời cảm ơn

Trong suốt quá trình thực hiện và hoàn thành đồ án chuyên ngành, chúng em đã nhận được rất nhiều sự hướng dẫn, hỗ trợ và giúp đỡ quý báu từ các thầy cô, bạn bè và gia đình.

Trước hết, chúng em xin bày tỏ lòng biết ơn sâu sắc đến quý Thầy hướng dẫn: PGS. TS. Lê Hồng Trang và ThS. Dương Đức Tín. Những định hướng chuyên môn, sự góp ý tận tình và đồng hành của các Thầy trong suốt quá trình từ đề xuất hướng tiếp cận đến khi thử nghiệm và đánh giá thành công hệ thống chính là tiền đề quan trọng giúp đồ án được hoàn thiện.

Chúng em cũng xin chân thành cảm ơn Quý Thầy Cô thuộc Khoa Khoa học & Kỹ thuật Máy tính, Trường Đại học Bách khoa - ĐHQG TP.HCM. Những kiến thức nền tảng và chuyên sâu mà Thầy Cô truyền đạt trong suốt những năm tháng trên giảng đường chính là hành trang vững chắc để chúng em có thể thực hiện thành công đề tài này.

Bên cạnh đó, chúng em xin gửi lời cảm ơn đến gia đình và bạn bè, những người đã luôn động viên, khích lệ tinh thần và tạo mọi điều kiện thuận lợi nhất trong suốt quá trình học tập.

Cuối cùng, dù đã nỗ lực hết sức, báo cáo khó tránh khỏi những thiếu sót nhất định. Chúng em rất mong nhận được những ý kiến đóng góp quý báu từ Quý Thầy Cô Hội đồng để đề tài được hoàn thiện hơn, đồng thời rút ra những bài học kinh nghiệm cho các chặng đường nghiên cứu và học tập tiếp theo.

Tóm tắt

Đề án này nghiên cứu bài toán phát hiện bất thường liên tục (Continual Anomaly Detection – CAD) trên ảnh công nghiệp. Trong thiết lập này, dữ liệu không xuất hiện một lần dưới dạng tập huấn luyện cố định, mà được tiếp nhận theo từng nhóm sản phẩm hoặc từng tác vụ theo thời gian. Thách thức chính là hệ thống phải cập nhật tri thức từ dữ liệu mới nhưng vẫn hạn chế suy giảm hiệu năng trên các tác vụ đã học trước đó.

Đề tài tiếp cận bài toán theo hướng Học lồng nhau (Nested Learning), trong đó quá trình học được tổ chức thành nhiều tầng bộ nhớ có tốc độ cập nhật khác nhau. Hệ thống sử dụng DINOv2 làm backbone trích xuất đặc trưng ảnh, kết hợp tầng bộ nhớ cập nhật nhanh để tiếp nhận các biến đổi mới trong không gian embedding và tầng bộ nhớ ổn định dựa trên incremental coreset để lưu giữ các đại diện bình thường quan trọng. Thay vì huấn luyện lại toàn bộ mô hình hoặc lưu trữ toàn bộ dữ liệu lịch sử, phương pháp tập trung vào cập nhật bộ nhớ đặc trưng một cách có kiểm soát.

Bên cạnh bộ nhớ chính, đề án xây dựng quy trình củng cố offline gồm utility-guided anchor replay và metric-gated acceptance. Quy trình này chọn các mẫu đại diện có độ hữu dụng cao để củng cố biểu diễn bình thường, đồng thời chỉ giữ lại phiên bản mới khi các chỉ số tích lũy không suy giảm vượt ngưỡng cho phép. Thực nghiệm được thực hiện trên MVTec AD theo thiết lập 15 tác vụ tuần tự, với đánh giá ở cả cấp ảnh và cấp điểm ảnh. Kết quả cho thấy hướng tiếp cận đề xuất đạt hiệu năng cạnh tranh trong thiết lập học liên tục, hạn chế suy giảm trên các tác vụ cũ và vẫn kiểm soát được dung lượng bộ nhớ thông qua coreset.

Bảng chú giải

Thuật ngữ	Giải thích
ACC	Adaptive Consistency Check, cơ chế kiểm tra độ nhất quán giữa biểu diễn gốc và biểu diễn sau thích nghi trước khi cập nhật bộ nhớ ổn định
anchor replay	Cơ chế sử dụng lại một số mẫu đại diện được chọn từ bộ nhớ hoặc coreset để hỗ trợ củng cố mô hình
AUPR	Area Under the Precision-Recall Curve, chỉ số đánh giá chất lượng xếp hạng trong bối cảnh dữ liệu mất cân bằng
AUROC	Area Under the Receiver Operating Characteristic Curve, chỉ số đánh giá khả năng xếp hạng mẫu dương cao hơn mẫu âm trên nhiều ngưỡng
backbone	Mô hình nền dùng để trích xuất đặc trưng từ dữ liệu đầu vào; trong thị giác máy tính, backbone thường là một mạng đã được huấn luyện trước và cung cấp embedding cho các module xử lý phía sau
CAD	Continual Anomaly Detection, bài toán phát hiện bất thường trong thiết lập học liên tục, nơi mô hình phải học qua nhiều tác vụ hoặc phân phối dữ liệu theo thời gian
CADIC	Continual Anomaly Detection Based on Incremental Coreset, hướng phát hiện bất thường liên tục dựa trên coreset tăng dần
catastrophic forgetting	Hiện tượng mô hình suy giảm đáng kể hiệu năng trên tri thức hoặc tác vụ cũ sau khi được cập nhật trên dữ liệu mới
CBP	Continual Backpropagation, cơ chế theo dõi utility và tái khởi tạo các đơn vị kém hoạt động để duy trì khả năng thích nghi lâu dài

Thuật ngữ	Giải thích
CMS	Continuum Memory System, cơ chế bộ nhớ liên tục dùng để lưu giữ và cập nhật các biểu diễn đại diện trong quá trình học liên tục
CNN	Convolutional Neural Network (Mạng nơ-ron tích chập), kiến trúc mạng học sâu sử dụng các phép toán tích chập để trích xuất các đặc trưng cục bộ, là một kiến trúc mạng học sâu chuyên dụng để trích xuất đặc trưng và xử lý dữ liệu không gian như hình ảnh
coreset	Tập con đại diện của các embedding hoặc đặc trưng patch, được chọn nhằm xấp xỉ phân phối của tập đặc trưng đầy đủ trong khi giảm kích thước bộ nhớ và chi phí tìm kiếm
data drift	Sự dịch chuyển dữ liệu theo thời gian trong quá trình vận hành, có thể do thay đổi môi trường, thiết bị, sản phẩm hoặc quy trình
DINOv2	Mô hình thị giác tự giám sát dùng để trích xuất đặc trưng tổng quát ở cấp độ ảnh và patch
distribution shift	Sự thay đổi phân phối dữ liệu giữa các giai đoạn huấn luyện, kiểm thử hoặc vận hành thực tế
Fast Memory	Tầng bộ nhớ cập nhật nhanh, dùng để hấp thụ các thay đổi ngắn hạn trong biểu diễn dữ liệu
fine-tuning	Quá trình tinh chỉnh một mô hình đã được huấn luyện trước trên một tập dữ liệu hoặc tác vụ cụ thể, thường nhằm thích nghi mô hình với miền dữ liệu mới mà không cần huấn luyện lại từ đầu
Forgetting Measure	Chỉ số đánh giá mức suy giảm hiệu năng trên các tác vụ cũ sau khi mô hình học thêm tác vụ mới
LEJEP	Surrogate loss lấy cảm hứng từ họ Joint-Embedding Predictive Architecture, dùng để dự đoán hoặc bảo toàn biểu diễn trong không gian tiềm ẩn
metric-gated acceptance	Cơ chế chỉ chấp nhận checkpoint mới nếu các chỉ số đánh giá tích lũy không suy giảm vượt quá ngưỡng cho phép



Thuật ngữ	Giải thích
MVTec AD	Bộ dữ liệu phát hiện bất thường công nghiệp gồm nhiều loại sản phẩm, ảnh bình thường, ảnh bất thường và mặt nạ lỗi
Nested Learning	Hướng tiếp cận tổ chức quá trình học thành nhiều tầng hoặc nhiều vòng tối ưu lồng nhau, mỗi tầng có vai trò và tốc độ cập nhật khác nhau
NSP2	Null-Space Projection, cơ chế chiếu gradient nhằm giảm ảnh hưởng của cập nhật mới lên các hướng biểu diễn liên quan đến tri thức cũ
plasticity	Khả năng của mô hình trong việc tiếp thu tri thức mới hoặc thích nghi với phân phối dữ liệu mới
ReplayCAD	Phương pháp phát hiện bất thường liên tục sử dụng diffusion-based generative replay để phát lại dữ liệu lịch sử có chi tiết pixel
Subspace Recycling	Cơ chế tái sử dụng hoặc giải phóng một phần không gian con khi null space cho cập nhật an toàn trở nên quá nhỏ
TITANS	Hướng tiếp cận học bộ nhớ tại thời điểm kiểm thử, trong đó mô hình duy trì cơ chế ghi nhớ có thể cập nhật khi gặp ngữ cảnh mới
TTA	Test-Time Adaptation, cơ chế cho phép mô hình thích nghi tại thời điểm suy luận hoặc kiểm thử mà không cần huấn luyện lại toàn bộ
utility score	Điểm đánh giá mức độ hữu ích hoặc giá trị đại diện của một phần tử trong bộ nhớ khi chọn mẫu anchor hoặc cập nhật coreset
Vision Transformer	Kiến trúc Transformer cho thị giác máy tính, trong đó ảnh được chia thành các patch và xử lý bằng cơ chế self-attention

Mục lục

1	Giới thiệu đề tài	1
1.1	Đặt vấn đề	1
1.2	Mục tiêu	2
1.3	Phạm vi	3
1.4	Cấu trúc báo cáo	3
2	Các nghiên cứu liên quan	5
2.1	Phát hiện bất thường trong ảnh công nghiệp	5
2.2	Các phương pháp dựa trên đặc trưng và ngân hàng bộ nhớ	6
2.2.1	PaDiM	7
2.2.2	PatchCore	8
2.3	Học liên tục và hiện tượng quên thảm khốc	10
2.3.1	Học liên tục	10
2.3.2	Mâu thuẫn nội tại trong học liên tục	11
2.3.3	Hiện tượng quên thảm khốc	13
2.3.4	Các hướng tiếp cận trong học liên tục	13
2.4	Phát hiện bất thường liên tục	17
2.4.1	Tổng quan	17
2.4.2	Các hướng tiếp cận giải quyết bài toán phát hiện bất thường liên tục	18
2.5	Học lồng nhau	21
2.5.1	Khái niệm Học lồng nhau	21
2.5.2	Mô hình như một hệ tối ưu nhiều tầng	22
2.5.3	Bộ nhớ đa thang thời gian (multi-timescale)	23
2.6	Mô hình thị giác nền tảng cho trích xuất đặc trưng ảnh	23
3	Cơ sở lý thuyết	25
3.1	Biểu diễn đặc trưng trong phát hiện bất thường công nghiệp	25
3.1.1	Biểu diễn ảnh bằng patch embedding	25
3.1.2	Điểm bất thường trong không gian đặc trưng	26
3.2	Bộ nhớ đặc trưng và coreset trong phát hiện bất thường liên tục	27
3.2.1	Ngân hàng bộ nhớ đặc trưng	27
3.2.2	Cập nhật incremental coreset	27
3.3	Thiết lập học liên tục cho phát hiện bất thường	29

3.3.1	Chuỗi tác vụ và dữ liệu bình thường	29
3.3.2	Cập nhật bộ nhớ qua nhiều tác vụ	29
3.4	Cơ sở học lồng nhau cho mô hình đề xuất	30
3.4.1	Hệ học nhiều tầng	30
3.4.2	Bộ nhớ cập nhật theo nhiều thang thời gian	30
3.5	Chỉ số đánh giá	32
3.5.1	Đánh giá cấp ảnh và cấp điểm ảnh	32
3.5.2	Đánh giá mức độ quên	34
4	Giải pháp đề xuất	36
4.1	Tổng quan phương pháp	36
4.2	Trích xuất biểu diễn ảnh bằng Vision Backbone	38
4.3	Cơ chế Fast Memory TITANS và Test-Time Adaptation	39
4.4	ACC Gating và kiểm soát cập nhật bộ nhớ	40
4.5	CADIC coreset và bộ nhớ ổn định	41
4.6	Utility-guided anchor replay	42
4.7	Tính điểm bất thường bằng patch nearest-neighbor	43
4.8	Phase 3: củng cố offline có kiểm soát	44
4.9	Vai trò của NSP2 và CBP trong thiết kế mở rộng	45
5	Thực nghiệm	47
5.1	Mục tiêu thực nghiệm	47
5.2	Bộ dữ liệu	47
5.3	Cấu hình thực nghiệm	48
5.4	Chỉ số đánh giá	49
5.5	Kết quả main reportable benchmark trên 15 tác vụ	50
5.6	Kết quả experimental max-power với NSP2 và CBP	50
5.7	Kiểm thử cơ chế	52
5.8	Thảo luận kết quả thực nghiệm	52
6	Đánh giá và thảo luận	54
6.1	Đánh giá kiến trúc đề xuất	54
6.2	Ba cơ chế bảo vệ tri thức cũ	55
6.3	So sánh định tính với hướng replay-based CAD	56
6.4	Hạn chế	56



6.5	Hướng phát triển	57
7	Tổng kết	60
	Tài liệu tham khảo	62

Danh sách hình vẽ

1	Ảnh minh họa cho tập dữ liệu MVTEC AD. Hàng trên là 5 loại kết cấu (Textures), các hàng dưới là 10 loại vật thể (Objects). Các cột bên trái màu xanh lá là ảnh không lỗi, các cột bên phải màu đỏ là ảnh lỗi. Nguồn: theo Bergmann và cộng sự [1].	6
2	Minh họa quy trình PaDiM: mô hình trích xuất đặc trưng patch từ ảnh bình thường, ước lượng phân phối Gaussian tại từng vị trí không gian và tính điểm bất thường bằng khoảng cách Mahalanobis. Nguồn: theo Defard và cộng sự [2].	8
3	Minh họa quy trình của PatchCore. Ảnh bình thường được đưa qua backbone đã huấn luyện trước để trích xuất đặc trưng patch, sau đó các đặc trưng này được rút gọn bằng coresets và lưu vào memory bank. Khi kiểm thử, điểm bất thường được tính dựa trên khoảng cách lân cận gần nhất giữa đặc trưng ảnh kiểm thử và các đặc trưng bình thường trong memory bank. Nguồn: theo Roth và cộng sự [3].	9
4	Minh họa năm hướng tiếp cận tiêu biểu trong học liên tục hiện đại, bao gồm: điều chuẩn, phát lại dữ liệu, tối ưu hóa, biểu diễn và kiến trúc. Nguồn: theo Wang và cộng sự [4].	14
5	Sơ đồ bộ nhớ nhiều thang thời gian cho phát hiện bất thường liên tục. Tầng cập nhật nhanh hỗ trợ thích nghi với đặc trưng mới, trong khi tầng cập nhật chậm duy trì các đại diện ổn định dùng cho so khớp lân cận gần nhất.	22
6	Pipeline tổng thể của giải pháp CAD được đề xuất, gồm ba giai đoạn chính: Thích nghi nhanh, tiêu hóa tri thức và củng cố offline (Tiến hoá).	36
7	Lộ trình phát triển đề xuất cho Meta-NATH CAD. Các bước ngắn hạn tập trung vào việc khóa baseline và cải thiện cumulative pixel localization; các bước trung hạn đánh giá độc lập NSP2, CBP và Subspace Recycling; các bước dài hạn mở rộng sang VisA, DINOv3 và tối ưu triển khai cloud/industrial inference.	59



Danh sách bảng

2	Kết quả trước và sau Phase 3 conservative trên 15 tác vụ MVTec AD.	50
3	Kết quả trước và sau Phase 3 max-power trên 15 tác vụ MVTec AD.	51
4	Kết quả kiểm thử các cơ chế chính trong pipeline.	52

Chương 1

Giới thiệu đề tài

1.1 Đặt vấn đề

Phát hiện bất thường trên ảnh công nghiệp là bài toán nhận diện các sản phẩm hoặc vùng ảnh có sai lệch so với trạng thái bình thường. Các sai lệch này có thể là trầy xước, nứt vỡ, biến dạng, khuyết tật bề mặt hoặc thay đổi bất thường về kết cấu. Trong môi trường sản xuất, việc phát hiện sớm các lỗi như vậy có ý nghĩa quan trọng vì chúng ảnh hưởng trực tiếp đến chất lượng sản phẩm và độ ổn định của dây chuyền.

Một đặc điểm khó của bài toán này là dữ liệu lỗi thường hiếm và khó thu thập đầy đủ. Trên thực tế, hệ thống thường có nhiều ảnh bình thường hơn ảnh lỗi, trong khi các dạng lỗi mới có thể chỉ xuất hiện sau một thời gian vận hành. Vì vậy, nhiều phương pháp phát hiện bất thường công nghiệp không học trực tiếp từng loại lỗi, mà học biểu diễn của dữ liệu bình thường và xem những mẫu lệch khỏi biểu diễn đó là bất thường.

Các phương pháp dựa trên đặc trưng thị giác và ngân hàng bộ nhớ đã cho thấy hiệu quả tốt trong bối cảnh này. Ảnh đầu vào được đưa qua một backbone đã huấn luyện trước để trích xuất embedding ở mức ảnh hoặc mức patch. Sau đó, các embedding bình thường được lưu trong bộ nhớ và dùng làm cơ sở so khớp khi kiểm thử. Nếu một patch hoặc một ảnh kiểm thử nằm xa các đặc trưng bình thường đã lưu, nó được xem là có khả năng bất thường cao. Cách tiếp cận này phù hợp với kiểm tra lỗi công nghiệp vì nhiều khuyết tật chỉ xuất hiện ở một vùng nhỏ trên ảnh.

Tuy nhiên, phần lớn các hệ thống phát hiện bất thường vẫn được xây dựng trong thiết lập tĩnh. Mô hình hoặc bộ nhớ đặc trưng thường được xây dựng một lần từ một tập dữ liệu cố định, sau đó được giả định sẽ hoạt động trên dữ liệu có phân phối tương tự. Giả định này chưa phản ánh đầy đủ điều kiện sản xuất thực tế. Trong quá trình vận hành, dây chuyền có thể tiếp nhận sản phẩm mới, điều kiện chiếu sáng hoặc góc chụp có thể thay đổi, thiết bị có thể bị hao mòn, và phân phối dữ liệu bình thường có thể dịch chuyển theo thời gian.

Khi dữ liệu mới xuất hiện, hệ thống cần được cập nhật. Nếu chỉ tinh chỉnh trực tiếp theo dữ liệu mới, mô hình có thể suy giảm hiệu năng trên các nhóm sản phẩm hoặc tác vụ cũ, dẫn đến hiện tượng quên thăm khốc. Ngược lại, nếu huấn luyện lại toàn bộ mô hình với

dữ liệu lịch sử sau mỗi lần cập nhật, chi phí tính toán, thời gian huấn luyện và dung lượng lưu trữ sẽ tăng đáng kể. Các hướng dùng replay hoặc memory bank có thể giảm bớt vấn đề này, nhưng vẫn gặp các đánh đổi về bộ nhớ, chi phí cập nhật hoặc khả năng mở rộng khi số lượng tác vụ tăng lên.

Từ những vấn đề trên, phát hiện bất thường liên tục (Continual Anomaly Detection – CAD) là một hướng tiếp cận phù hợp cho các hệ thống kiểm tra lỗi công nghiệp trong điều kiện dữ liệu thay đổi theo thời gian. Trong thiết lập này, hệ thống cần tiếp nhận các tác vụ hoặc nhóm dữ liệu mới mà không làm mất các đại diện quan trọng của dữ liệu đã học. Đề tài này tiếp cận CAD theo hướng Học lồng nhau (Nested Learning), trong đó bộ nhớ đặc trưng được tổ chức thành nhiều tầng với tốc độ cập nhật khác nhau, nhằm cân bằng giữa khả năng thích nghi và khả năng duy trì tri thức cũ.

1.2 Mục tiêu

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống phát hiện bất thường liên tục cho ảnh công nghiệp dựa trên đặc trưng thị giác và bộ nhớ phân tầng. Hệ thống hướng đến khả năng cập nhật bộ nhớ theo từng tác vụ, kiểm soát dung lượng lưu trữ và duy trì hiệu năng phát hiện bất thường trong quá trình học tuần tự.

Cụ thể, đề tài tập trung vào các mục tiêu sau:

- Nghiên cứu bài toán phát hiện bất thường công nghiệp trong thiết lập học liên tục, trong đó dữ liệu được tiếp nhận theo chuỗi tác vụ hoặc nhóm sản phẩm.
- Khảo sát các hướng tiếp cận liên quan đến phát hiện bất thường, học liên tục, hiện tượng quên thảm khốc, phát hiện bất thường liên tục và Học lồng nhau.
- Xây dựng pipeline phát hiện bất thường dựa trên backbone thị giác DINOv2 và không gian embedding, thay vì huấn luyện một bộ phân loại lỗi từ đầu.
- Thiết kế cơ chế bộ nhớ phân tầng gồm tầng cập nhật nhanh và tầng bộ nhớ ổn định dựa trên incremental coreset, nhằm cân bằng giữa khả năng thích nghi với dữ liệu mới và khả năng duy trì tri thức cũ.
- Triển khai quy trình củng cố offline có kiểm soát, kết hợp utility-guided anchor replay và metric-gated acceptance để hỗ trợ cập nhật bộ nhớ mà không làm suy giảm mạnh các chỉ số đánh giá tích lũy.
- Đánh giá hệ thống trên MVTec AD theo thiết lập nhiều tác vụ tuần tự, sử dụng các

chỉ số ở cấp ảnh, cấp điểm ảnh và mức độ suy giảm hiệu năng qua quá trình học liên tục.

1.3 Phạm vi

Trong phạm vi đồ án, bài toán được giới hạn ở phát hiện bất thường trên ảnh công nghiệp. Dữ liệu đầu vào là ảnh sản phẩm hoặc ảnh bề mặt công nghiệp; đề tài không xét các dạng dữ liệu phi ảnh như tín hiệu cảm biến, âm thanh hoặc chuỗi thời gian.

Về loại bất thường, đề tài tập trung vào các sai lệch có thể quan sát trực tiếp trên ảnh, chẳng hạn như trầy xước, nứt vỡ, biến dạng, khuyết tật bề mặt hoặc sai lệch kết cấu. Bài toán được đánh giá ở hai mức: phát hiện ảnh bất thường và định vị vùng bất thường khi dữ liệu có mặt nạ lỗi. Đề tài không đặt trọng tâm vào phân loại chi tiết từng loại lỗi theo nhãn ngữ nghĩa.

Về dữ liệu thực nghiệm, MVTec AD được sử dụng làm bộ dữ liệu chính do có nhiều nhóm vật thể và kết cấu công nghiệp, bao gồm ảnh bình thường, ảnh bất thường và mặt nạ lỗi ở mức điểm ảnh [1]. Trong thiết lập học liên tục, các nhóm dữ liệu được tổ chức thành chuỗi tác vụ để đánh giá khả năng cập nhật và duy trì hiệu năng của hệ thống.

Về mô hình, đồ án không tập trung vào việc huấn luyện một backbone thị giác mới từ đầu. Trọng tâm của đề tài là cơ chế tổ chức bộ nhớ, cập nhật bộ nhớ qua nhiều tác vụ, chọn mẫu đại diện bằng coresets và củng cố offline có kiểm soát. Vì vậy, phạm vi nghiên cứu không nằm ở việc tối ưu toàn bộ mô hình thị giác, mà nằm ở cách khai thác đặc trưng đã học sẵn để duy trì khả năng phát hiện bất thường trong quá trình dữ liệu thay đổi theo thời gian.

Đề tài cũng chưa giải quyết đầy đủ các vấn đề triển khai công nghiệp ở mức hệ thống hoàn chỉnh như tích hợp camera, xử lý dữ liệu thời gian thực trên dây chuyền, tối ưu tốc độ suy luận trên mọi phần cứng hoặc xây dựng giao diện vận hành.

1.4 Cấu trúc báo cáo

Báo cáo được tổ chức thành bảy chương như sau:

- **Chương 1: Giới thiệu đề tài** – Trình bày bối cảnh nghiên cứu, vấn đề cần giải quyết, mục tiêu và phạm vi của đồ án.
- **Chương 2: Các nghiên cứu liên quan** – Tổng hợp các hướng nghiên cứu về phát

hiện bất thường công nghiệp, phương pháp dựa trên đặc trưng và memory bank, học liên tục, phát hiện bất thường liên tục, học lồng nhau và mô hình thị giác nền tảng.

- **Chương 3: Cơ sở lý thuyết** – Trình bày các khái niệm và công thức được sử dụng trực tiếp trong đồ án, bao gồm biểu diễn đặc trưng, điểm bất thường, coreset, thiết lập học liên tục, cơ sở học lồng nhau và các chỉ số đánh giá.
- **Chương 4: Giải pháp đề xuất** – Mô tả kiến trúc và pipeline của hệ thống, bao gồm trích xuất đặc trưng bằng backbone thị giác, Fast Memory, ACC gating, CADIC coreset, utility-guided anchor replay, tính điểm bất thường và quy trình củng cố offline.
- **Chương 5: Thực nghiệm** – Trình bày bộ dữ liệu, cấu hình thực nghiệm, kịch bản học tuần tự, chỉ số đánh giá và kết quả thực nghiệm trên MVTec AD.
- **Chương 6: Đánh giá và thảo luận** – Phân tích kết quả đạt được, vai trò của các thành phần chính, so sánh định tính với một số hướng liên quan, nêu hạn chế và hướng phát triển.
- **Chương 7: Tổng kết** – Tóm tắt nội dung chính của đồ án, các kết quả đạt được và định hướng tiếp tục hoàn thiện hệ thống.

Chương 2

Các nghiên cứu liên quan

2.1 Phát hiện bất thường trong ảnh công nghiệp

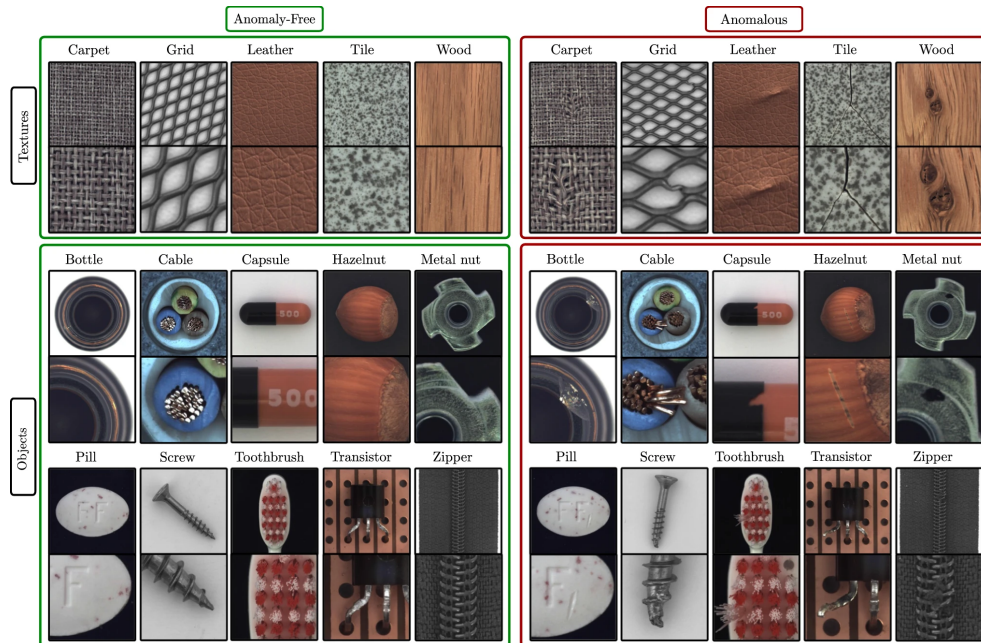
Phát hiện bất thường là bài toán nhận dạng các mẫu dữ liệu có hành vi khác biệt đáng kể so với phần lớn dữ liệu bình thường hoặc không phù hợp với hành vi được kỳ vọng của hệ thống [5]. Trong bối cảnh thị giác máy tính, bài toán này thường được đặt ra dưới dạng phát hiện các cấu trúc, vùng ảnh hoặc mẫu hình có sự sai khác so với phân phối của dữ liệu bình thường. Khác với bài toán phân loại thông thường, nơi các lớp mục tiêu thường được định nghĩa rõ ràng và có dữ liệu huấn luyện tương ứng, phát hiện bất thường thường phải xử lý tình huống dữ liệu lỗi hiếm, khó dự đoán và không thể thu thập đầy đủ từ trước.

Trong công nghiệp, việc duy trì chất lượng sản phẩm là vấn đề sống còn của mỗi doanh nghiệp. Khi đó, bài toán phát hiện bất thường trên ảnh được ứng dụng rộng rãi trong hệ thống kiểm tra lỗi ngoại quan và giám sát chất lượng sản phẩm. Các lỗi như trầy xước, nứt vỡ, móp méo, bẩn bề mặt, sai lệch kết cấu hoặc thiếu thành phần có thể ảnh hưởng trực tiếp đến chất lượng đầu ra của dây chuyền sản xuất. Do đó, hệ thống phát hiện bất thường công nghiệp không chỉ cần phân biệt ảnh bình thường và ảnh lỗi, mà trong nhiều trường hợp còn cần định vị vùng bất thường ở mức điểm ảnh để hỗ trợ phân tích nguyên nhân và kiểm soát quy trình sản xuất [6].

Một đặc điểm quan trọng của phát hiện bất thường công nghiệp là dữ liệu lỗi thường hiếm và đa dạng. Trong nhiều môi trường sản xuất, việc thu thập đầy đủ tất cả các loại lỗi là không khả thi vì lỗi có thể xuất hiện với tần suất thấp, hình thái không ổn định hoặc thay đổi theo điều kiện vận hành. Vì vậy, nhiều phương pháp được xây dựng trong thiết lập không giám sát hoặc bán giám sát, trong đó mô hình chủ yếu học từ ảnh bình thường và xem các mẫu lệch khỏi phân phối bình thường là bất thường. Thiết lập này phù hợp với thực tế công nghiệp, nơi dữ liệu bình thường thường dễ thu thập hơn dữ liệu lỗi.

MVTec AD là một trong những bộ dữ liệu chuẩn được sử dụng phổ biến cho bài toán phát hiện bất thường công nghiệp. Bộ dữ liệu này được thiết kế nhằm mô phỏng các kịch bản kiểm tra công nghiệp trong thực tế, bao gồm nhiều nhóm vật thể và kết cấu, với tập huấn luyện chủ yếu gồm ảnh không lỗi và tập kiểm thử gồm cả ảnh bình thường lẫn ảnh có lỗi [1]. Ngoài nhãn ảnh, MVTEC AD còn cung cấp mặt nạ vùng lỗi ở mức điểm ảnh, cho

phép đánh giá cả bài toán phát hiện ảnh bất thường và bài toán định vị vùng bất thường.



Hình 1: Ảnh minh họa cho tập dữ liệu MVTEC AD. Hàng trên là 5 loại kết cấu (Textures), các hàng dưới là 10 loại vật thể (Objects). Các cột bên trái màu xanh lá là ảnh không lỗi, các cột bên phải màu đỏ là ảnh lỗi. Nguồn: theo Bergmann và cộng sự [1].

Từ các đặc điểm trên, có thể thấy bài toán phát hiện bất thường trong ảnh công nghiệp đặt ra yêu cầu phải vừa xác định ảnh lỗi và vừa biểu diễn được sự sai khác cục bộ giữa vùng bình thường và vùng bất thường. Vì vậy, chất lượng của đặc trưng ảnh và cách tổ chức các đặc trưng bình thường đóng vai trò quan trọng trong hiệu quả phát hiện. Đây là cơ sở để các phương pháp dựa trên biểu diễn đặc trưng và ngân hàng bộ nhớ (memory bank) được nghiên cứu rộng rãi trong lĩnh vực này [6].

2.2 Các phương pháp dựa trên đặc trưng và ngân hàng bộ nhớ

Sau khi các mô hình thị giác học sâu phát triển, nhiều phương pháp phát hiện bất thường công nghiệp không còn huấn luyện toàn bộ mô hình từ đầu mà tận dụng các backbone đã được huấn luyện trước để trích xuất đặc trưng ảnh. Ý tưởng chính của nhóm phương pháp này là chuyển ảnh đầu vào sang một không gian biểu diễn giàu ngữ nghĩa hơn, trong đó các mẫu bình thường có xu hướng tạo thành những cụm hoặc phân phối ổn định. Khi kiểm thử, mức độ bất thường của một ảnh hoặc một vùng ảnh được đánh giá thông qua độ lệch của đặc trưng tương ứng so với các đặc trưng bình thường đã học.

Điểm đáng chú ý của hướng tiếp cận này là bài toán phát hiện bất thường được chuyển từ

không gian ảnh gốc sang không gian đặc trưng. Thay vì so sánh trực tiếp các giá trị điểm ảnh, mô hình so sánh các embedding được trích xuất từ những vùng ảnh nhỏ, hay còn gọi là patch. Cách làm này giúp hệ thống khai thác được cả thông tin ngữ nghĩa của đối tượng lẫn các sai lệch cục bộ trên bề mặt, vốn rất quan trọng trong các bài toán kiểm tra lỗi công nghiệp.

Nhìn chung, các phương pháp thuộc nhóm này có thể được chia thành hai hướng tiêu biểu. Hướng thứ nhất mô hình hóa phân phối của đặc trưng bình thường, chẳng hạn như ước lượng phân phối Gaussian tại từng vị trí không gian. Hướng thứ hai lưu trữ các đặc trưng bình thường trong một memory bank và đánh giá mẫu kiểm thử bằng khoảng cách đến các đặc trưng đã lưu. PaDiM và PatchCore là hai đại diện điển hình cho hai hướng tiếp cận này.

2.2.1 PaDiM

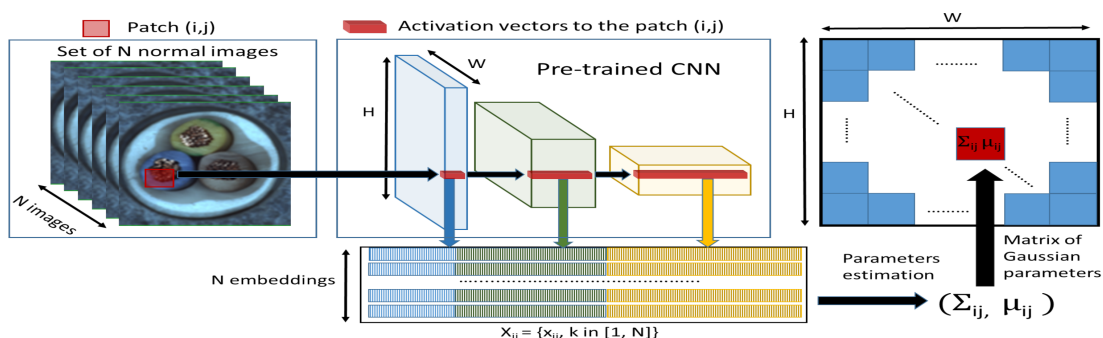
PaDiM là một phương pháp tiêu biểu cho hướng mô hình hóa phân phối đặc trưng trong bài toán phát hiện và định vị bất thường. Phương pháp này sử dụng một mạng CNN đã được huấn luyện trước để trích xuất embedding tại từng vị trí không gian của ảnh. Với mỗi vị trí patch, PaDiM tổng hợp các vector đặc trưng tương ứng từ toàn bộ tập ảnh bình thường, sau đó ước lượng một phân phối Gaussian đa biến cho vị trí đó [2].

Gọi $\mathbf{x}_{i,j}$ là vector đặc trưng tại vị trí không gian (i, j) của ảnh kiểm thử. Từ tập dữ liệu bình thường, PaDiM ước lượng vector trung bình $\mu_{i,j}$ và ma trận hiệp phương sai $\Sigma_{i,j}$ tại từng vị trí. Điểm bất thường tại tọa độ (i, j) được xác định thông qua khoảng cách Mahalanobis [2]:

$$s_{i,j} = \sqrt{(\mathbf{x}_{i,j} - \mu_{i,j})^T \Sigma_{i,j}^{-1} (\mathbf{x}_{i,j} - \mu_{i,j})}.$$

Giá trị $s_{i,j}$ càng lớn cho thấy đặc trưng tại vị trí đó càng lệch khỏi phân phối bình thường đã học. Bản đồ bất thường được xây dựng bằng cách tính điểm số này trên toàn bộ lưới không gian và nội suy về kích thước ảnh gốc. Điểm bất thường ở cấp độ ảnh thường được xác định bằng giá trị lớn nhất hoặc một phép tổng hợp trên bản đồ bất thường.

Ưu điểm của PaDiM là không cần dữ liệu lỗi trong quá trình huấn luyện và vẫn có thể tạo bản đồ bất thường ở mức điểm ảnh. Việc mô hình hóa phân phối đặc trưng tại từng vị trí không gian giúp phương pháp này khai thác được cấu trúc cục bộ của ảnh, vốn rất quan trọng trong kiểm tra lỗi bề mặt. Bên cạnh đó, do sử dụng đặc trưng từ mạng đã huấn



Hình 2: Minh họa quy trình PaDiM: mô hình trích xuất đặc trưng patch từ ảnh bình thường, ước lượng phân phối Gaussian tại từng vị trí không gian và tính điểm bất thường bằng khoảng cách Mahalanobis. Nguồn: theo Defard và cộng sự [2].

luyện trước, PaDiM có thể tận dụng các biểu diễn thị giác giàu thông tin mà không cần huấn luyện một mô hình phát hiện lỗi từ đầu.

Tuy nhiên, PaDiM vẫn tồn tại một số hạn chế. Thứ nhất, giả định đặc trưng bình thường tại mỗi vị trí tuân theo phân phối Gaussian đa biến có thể chưa đủ linh hoạt để mô tả các phân phối phức tạp trong dữ liệu hình ảnh công nghiệp [7]. Thứ hai, phương pháp cần ước lượng và lưu trữ vector trung bình cùng ma trận hiệp phương sai cho từng vị trí không gian, dẫn đến chi phí bộ nhớ tăng khi số chiều đặc trưng hoặc số lượng vị trí patch lớn [2]. Ngoài ra, PaDiM chủ yếu được thiết kế cho thiết lập tĩnh, trong đó tập dữ liệu bình thường được giả định là cố định. Khi dữ liệu mới xuất hiện theo thời gian, việc cập nhật các phân phối đặc trưng mà không làm ảnh hưởng đến biểu diễn đã học trước đó là một vấn đề chưa được giải quyết trực tiếp trong phương pháp này.

Những hạn chế trên là một trong các lý do thúc đẩy các hướng tiếp cận dựa trên so khớp đặc trưng và chọn mẫu đại diện, tiêu biểu là PatchCore [3].

2.2.2 PatchCore

Khác với định hướng mô hình hóa tham số của PaDiM, PatchCore tiếp cận bài toán phát hiện bất thường bằng cách lưu giữ trực tiếp các đặc trưng cục bộ của ảnh bình thường trong một memory bank. Phương pháp này sử dụng một backbone thị giác đã được huấn luyện trước để trích xuất đặc trưng ở mức patch, sau đó xây dựng memory bank từ các patch embedding của tập ảnh không lỗi. Trong giai đoạn suy luận, đặc trưng của ảnh kiểm thử được so khớp với các đặc trưng bình thường đã lưu bằng tìm kiếm lân cận gần nhất, từ đó xác định mức độ bất thường ở từng vùng ảnh [3].

Gọi $\mathcal{M}$ là memory bank chứa các đặc trưng patch bình thường, và $\mathbf{z}$ là vector đặc trưng

của một patch trong ảnh kiểm thử. Điểm bất thường cục bộ của patch đó được xác định bằng khoảng cách Euclid đến đặc trưng bình thường gần nhất trong $\mathcal{M}$:

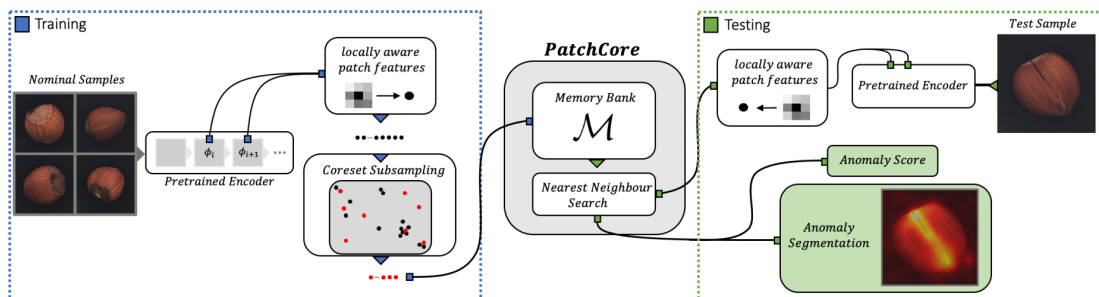
$$s(\mathbf{z}) = \min_{\mathbf{m} \in \mathcal{M}} \|\mathbf{z} - \mathbf{m}\|_2.$$

Giá trị $s(\mathbf{z})$ càng lớn cho thấy đặc trưng $\mathbf{z}$ càng nằm xa vùng phân bố của dữ liệu bình thường, do đó patch tương ứng có khả năng bất thường cao hơn. Bản đồ bất thường của toàn bộ ảnh được xây dựng bằng cách ánh xạ các điểm số này về lưới không gian của ảnh, trong khi điểm bất thường ở cấp độ ảnh thường được suy ra từ các vùng có điểm số cao nhất.

Đóng góp của PatchCore nằm ở cách tổ chức và thu gọn memory bank. Nếu lưu toàn bộ đặc trưng patch từ tập huấn luyện, kích thước bộ nhớ và chi phí tìm kiếm lân cận gần nhất sẽ tăng nhanh theo số lượng ảnh bình thường. Để khắc phục vấn đề này, PatchCore sử dụng coreset subsampling dựa trên bài toán MinMax Facility Location nhằm chọn một tập con đặc trưng có tính đại diện cao cho phân phối bình thường [3]. Một cách khái quát, mục tiêu chọn coreset có thể được biểu diễn như sau:

$$\mathcal{M}^* = \arg \min_{\mathcal{M} \subset \mathcal{M}_{\text{full}}, |\mathcal{M}| \leq K} \max_{\mathbf{x} \in \mathcal{M}_{\text{full}}} \min_{\mathbf{m} \in \mathcal{M}} \|\mathbf{x} - \mathbf{m}\|_2.$$

Trong đó, $\mathcal{M}_{\text{full}}$ là tập đầy đủ các đặc trưng patch bình thường, còn $\mathcal{M}^*$ là tập con được chọn sao cho mọi đặc trưng trong bộ nhớ đầy đủ đều được xấp xỉ bởi một phần tử gần nhất trong coreset. Nhờ đó, PatchCore giảm chi phí lưu trữ và suy luận, trong khi vẫn giữ được khả năng bao phủ không gian đặc trưng của dữ liệu bình thường.



Hình 3: Minh họa quy trình của PatchCore. Ảnh bình thường được đưa qua backbone đã huấn luyện trước để trích xuất đặc trưng patch, sau đó các đặc trưng này được rút gọn bằng coreset và lưu vào memory bank. Khi kiểm thử, điểm bất thường được tính dựa trên khoảng cách lân cận gần nhất giữa đặc trưng ảnh kiểm thử và các đặc trưng bình thường trong memory bank. Nguồn: theo Roth và cộng sự [3].

Ưu điểm của PatchCore là kết hợp được đặc trưng cục bộ từ backbone đã huấn luyện trước, cơ chế so khớp lân cận gần nhất và chiến lược nén bộ nhớ bằng coresets. So với PaDiM, PatchCore không cần giả định đặc trưng tại từng vị trí tuân theo phân phối Gaussian cố định, mà dựa trực tiếp vào cấu trúc hình học của không gian embedding. Nhờ vậy, phương pháp đạt hiệu quả cao trên các benchmark phát hiện bất thường công nghiệp như MVTec AD ở cả hai nhiệm vụ phát hiện ảnh bất thường và định vị vùng lỗi.

Tuy nhiên, PatchCore vẫn chủ yếu được thiết kế cho thiết lập tĩnh, trong đó memory bank được xây dựng một lần từ tập ảnh bình thường cố định. Khi chuyển sang bối cảnh dữ liệu công nghiệp thay đổi theo thời gian, việc cập nhật memory trở thành một vấn đề không đơn giản. Nếu chỉ liên tục bổ sung đặc trưng mới, hệ thống có thể gặp tình trạng tăng trưởng bộ nhớ theo thời gian; ngược lại, nếu rút gọn hoặc thay thế bộ nhớ không phù hợp, các đại diện quan trọng của phân phối cũ có thể bị loại bỏ. Các nghiên cứu về thích nghi liên tục trong phát hiện bất thường công nghiệp cũng chỉ ra rằng việc mở rộng mô hình phát hiện bất thường từ thiết lập tĩnh sang thiết lập học liên tục là cần thiết để thích nghi với dữ liệu mới mà không làm suy giảm tri thức đã học [8, 9]. Đó là một trong những lý do thúc đẩy việc mở rộng các phương pháp phát hiện bất thường tĩnh sang bài toán phát hiện bất thường liên tục (Continual Anomaly Detection).

2.3 Học liên tục và hiện tượng quên thảm khốc

2.3.1 Học liên tục

Học liên tục (continual learning; trong một số tài liệu còn được gọi là lifelong learning) là thiết lập trong đó mô hình không được huấn luyện một lần trên toàn bộ dữ liệu cố định, mà phải tiếp nhận dữ liệu theo từng giai đoạn, từng miền hoặc từng tác vụ. Về mặt cảm hứng sinh học, hướng nghiên cứu này liên hệ với khả năng học suốt đời của con người và động vật, khi hệ thần kinh có thể tiếp nhận kinh nghiệm mới mà không đánh mất hoàn toàn các kỹ năng hoặc ký ức đã hình thành trước đó [10]. Theo Wang và cộng sự, học liên tục có thể được xem là quá trình học từ các phân phối dữ liệu động, trong đó dữ liệu mới xuất hiện theo trình tự và dữ liệu cũ thường không còn được truy cập đầy đủ [4]. Do đó, bài toán đặt ra là phải vừa tối ưu cho học tri thức mới, vừa có khả năng duy trì tri thức cũ trong điều kiện dữ liệu lịch sử bị giới hạn.

Giả sử mô hình lần lượt học qua T tác vụ $\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_T$, trong đó mỗi tác vụ có phân phối dữ liệu riêng $\mathcal{D}_t$. Một cách lý tưởng, sau khi học đến tác vụ thứ t , mô hình cần duy trì hiệu năng trên toàn bộ các tác vụ đã gặp. Mục tiêu này có thể được mô tả khái quát

như sau:

$$\theta_t^* = \arg \min_{\theta} \sum_{k=1}^t \mathcal{L}_k(\theta; \mathcal{D}_k).$$

Tuy nhiên, trong thực tế học liên tục, dữ liệu của các tác vụ cũ thường bị giới hạn hoặc không còn khả dụng đầy đủ. Khi đó, quá trình cập nhật mô hình thường bị chi phối bởi dữ liệu của tác vụ hiện tại:

$$\theta_t = \arg \min_{\theta} \mathcal{L}_t(\theta; \mathcal{D}_t).$$

Sự khác biệt giữa hai mục tiêu trên cho thấy khó khăn cơ bản của học liên tục. Mô hình cần thích nghi với phân phối dữ liệu mới, nhưng quá trình thích nghi đó không được làm suy giảm nghiêm trọng hiệu năng trên các tác vụ đã học. Nếu không có cơ chế bảo toàn tri thức cũ, các cập nhật theo $\mathcal{D}_t$ có thể làm tham số mô hình dịch chuyển khỏi những nghiệm từng phù hợp với các tác vụ trước, từ đó dẫn đến hiện tượng quên thảm khốc.

2.3.2 Mâu thuẫn nội tại trong học liên tục

Khó khăn trên thường được diễn giải thông qua mâu thuẫn giữa tính ổn định và tính linh hoạt (stability – plasticity dilemma) [10]. Đây là hai yêu cầu cùng tồn tại trong học liên tục nhưng có xu hướng kéo quá trình cập nhật mô hình theo hai hướng khác nhau:

- **Tính linh hoạt (Plasticity):** là khả năng thay đổi của mô hình khi tiếp xúc với dữ liệu mới. Một mô hình có tính linh hoạt tốt có thể thích nghi với các phân phối mới, cập nhật biểu diễn khi xuất hiện tác vụ mới và mở rộng tri thức khi môi trường dữ liệu thay đổi. Trong học liên tục, tính linh hoạt giúp mô hình không bị giới hạn bởi những gì đã học trong quá khứ, từ đó có khả năng phản ứng với các mẫu dữ liệu chưa từng xuất hiện trước đó.
- **Tính ổn định (Stability):** là khả năng duy trì và bảo toàn các tri thức đã học. Một mô hình có tính ổn định tốt sẽ hạn chế việc các cập nhật mới phá vỡ những biểu diễn, tham số hoặc cấu trúc bộ nhớ quan trọng đối với các tác vụ trước đó. Trong học liên tục, tính ổn định giúp mô hình tránh việc học dữ liệu mới bằng cách ghi đè lên tri thức cũ.

Hai yêu cầu trên đều cần thiết, nhưng khó thỏa mãn đồng thời. Nếu mô hình quá ổn định, quá trình cập nhật sẽ bị hạn chế và mô hình khó thích nghi với dữ liệu mới. Ngược lại, nếu mô hình quá linh hoạt, các cập nhật trên dữ liệu hiện tại có thể làm thay đổi mạnh không gian biểu diễn đã học, khiến hiệu năng trên các tác vụ cũ suy giảm.

Trong một mạng nơ-ron, mâu thuẫn này thể hiện rõ ở việc nhiều tác vụ khác nhau có thể cùng sử dụng chung một tập tham số hoặc cùng chia sẻ một không gian biểu diễn. Khi mô hình học một tác vụ mới, các tham số được cập nhật để giảm mất mát trên dữ liệu hiện tại. Tuy nhiên, những tham số đó có thể cũng đang giữ vai trò quan trọng đối với các tác vụ đã học trước đó. Do đó, một cập nhật có lợi cho tác vụ mới chưa chắc có lợi cho tác vụ cũ. Nếu quá trình này diễn ra liên tục qua nhiều tác vụ mà không có cơ chế điều tiết, các biểu diễn cũ có thể dần bị thay thế bởi các biểu diễn mới.

Dưới góc độ tối ưu hóa, vấn đề trên có thể được nhìn nhận như sự xung đột giữa các mục tiêu học ở những thời điểm khác nhau. Ở mỗi giai đoạn học, mô hình được cập nhật theo hướng làm giảm hàm mất mát của dữ liệu hiện tại. Tuy nhiên, hướng cập nhật này có thể không tương thích với các mục tiêu học trước đó. Nói cách khác, gradient sinh ra từ tác vụ mới có thể làm tăng mất mát trên tác vụ cũ.

Trong bài toán phát hiện bất thường công nghiệp, mâu thuẫn ổn định–linh hoạt không chỉ xuất hiện ở tham số của mô hình, mà nó còn xuất hiện trong cách hệ thống biểu diễn trạng thái bình thường của dữ liệu. Khi dây chuyền sản xuất xuất hiện một nhóm sản phẩm mới hoặc điều kiện vận hành mới, hệ thống cần mở rộng không gian biểu diễn để bao phủ các mẫu bình thường mới. Tuy nhiên, nếu quá trình cập nhật làm thay đổi quá mạnh không gian đặc trưng hoặc bộ nhớ đại diện, các mẫu bình thường cũ có thể không còn được biểu diễn đầy đủ. Nói cách khác, nếu hệ thống không đủ linh hoạt, các sản phẩm hoặc điều kiện vận hành mới có thể bị nhận nhầm là bất thường, làm tăng số lượng cảnh báo sai. Ngược lại, nếu hệ thống cập nhật quá mạnh theo dữ liệu mới, các biểu diễn của nhóm sản phẩm cũ có thể bị suy giảm, khiến mô hình đánh mất khả năng phát hiện lỗi trên những tác vụ đã học. Do đó, trong phát hiện bất thường liên tục, mô hình cần đồng thời mở rộng được vùng biểu diễn của dữ liệu bình thường mới và duy trì được các đại diện quan trọng của dữ liệu bình thường cũ.

Ngoài mâu thuẫn ổn định – linh hoạt, các khảo sát gần đây còn nhấn mạnh rằng một hệ thống học liên tục tốt cần duy trì khả năng khái quát trong từng tác vụ và giữa các tác vụ, đồng thời vẫn đảm bảo hiệu quả về tài nguyên tính toán và bộ nhớ [4].

2.3.3 Hiện tượng quên thảm khốc

Quên thảm khốc (catastrophic forgetting) là hiện tượng mô hình sau khi học dữ liệu mới bị suy giảm đáng kể khả năng xử lý các dữ liệu hoặc tác vụ đã học trước đó. Vấn đề này đã được McCloskey và Cohen khảo sát từ rất sớm trong bối cảnh các mạng kết nối học tuần tự, cho thấy việc học một tập mẫu mới có thể gây nhiễu mạnh lên những tri thức đã được hình thành trước đó [11].

Trong các mô hình học sâu hiện đại, quên thảm khốc tiếp tục là một trở ngại quan trọng khi mô hình được huấn luyện qua nhiều tác vụ liên tiếp. Nguyên nhân chính là nhiều tác vụ khác nhau thường chia sẻ cùng một tập tham số hoặc cùng một không gian biểu diễn. Khi mô hình được cập nhật để giảm lỗi trên dữ liệu mới, các tham số quan trọng đối với tác vụ cũ có thể bị thay đổi, làm phá vỡ những biểu diễn đã học trước đó.

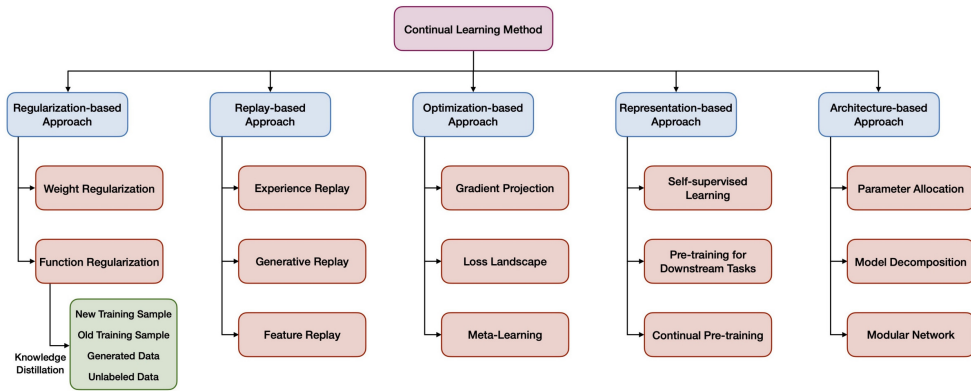
Một cách thường dùng để định lượng mức độ quên là so sánh hiệu năng tốt nhất từng đạt được trên một tác vụ cũ với hiệu năng của tác vụ đó sau khi mô hình đã học xong toàn bộ chuỗi tác vụ [12]. Gọi $a_{i,j}$ là hiệu năng của mô hình trên tác vụ $\mathcal{T}_j$ sau khi đã học xong tác vụ $\mathcal{T}_i$. Mức độ quên trung bình sau khi học xong T tác vụ có thể được mô tả như sau:

$$F_T = \frac{1}{T-1} \sum_{j=1}^{T-1} \left(\max_{i \in \{1, \dots, T-1\}} a_{i,j} - a_{T,j} \right).$$

Trong đó, $\max_{i \in \{1, \dots, T-1\}} a_{i,j}$ biểu diễn hiệu năng tốt nhất từng đạt được trên tác vụ $\mathcal{T}_j$ trước khi kết thúc toàn bộ quá trình học, còn $a_{T,j}$ là hiệu năng trên tác vụ đó sau khi mô hình đã học xong tác vụ cuối cùng. Giá trị F_T càng lớn cho thấy mô hình càng suy giảm mạnh trên các tác vụ cũ.

2.3.4 Các hướng tiếp cận trong học liên tục

Để hạn chế hiện tượng quên thảm khốc, các nghiên cứu về học liên tục đã đề xuất nhiều nhóm phương pháp khác nhau. Nhìn chung, các phương pháp này đều cố gắng giải quyết cùng một vấn đề: làm thế nào để mô hình tiếp thu tri thức mới mà không làm suy giảm nghiêm trọng hiệu năng trên các tri thức đã học. Theo khảo sát của Wang và cộng sự, các phương pháp học liên tục có thể được phân thành năm hướng chính, bao gồm: phương pháp dựa trên điều chuẩn, phương pháp dựa trên phát lại dữ liệu, phương pháp dựa trên tối ưu hóa, phương pháp dựa trên biểu diễn và phương pháp dựa trên kiến trúc [4].



Hình 4: Minh họa năm hướng tiếp cận tiêu biểu trong học liên tục hiện đại, bao gồm: điều chuẩn, phát lại dữ liệu, tối ưu hóa, biểu diễn và kiến trúc. Nguồn: theo Wang và cộng sự [4].

Phương pháp dựa trên điều chuẩn

Phương pháp này hạn chế sự thay đổi của mô hình khi học tác vụ mới bằng cách bổ sung các ràng buộc vào hàm mất mát. Ý tưởng chính là một số tham số, biểu diễn hoặc hành vi của mô hình có vai trò quan trọng đối với các tác vụ cũ, do đó không nên bị thay đổi quá mạnh khi mô hình tiếp nhận dữ liệu mới. Một cách tổng quát, hàm mất mát của nhóm phương pháp này có thể được viết dưới dạng:

$$\mathcal{L}(\theta) = \mathcal{L}_t(\theta) + \lambda \Omega(\theta, \theta^*),$$

trong đó $\mathcal{L}_t(\theta)$ là mất mát trên tác vụ hiện tại, $\Omega(\theta, \theta^*)$ là hạng điều chuẩn nhằm hạn chế mô hình lệch quá xa khỏi trạng thái đã học trước đó, và λ điều khiển mức độ ràng buộc. Một đại diện tiêu biểu là Elastic Weight Consolidation, trong đó mức độ quan trọng của tham số được ước lượng để bảo vệ các tham số quan trọng đối với tác vụ cũ [13].

Ưu điểm của nhóm điều chuẩn là không nhất thiết phải lưu trữ dữ liệu cũ, nhờ đó giảm áp lực về bộ nhớ và phù hợp với các tình huống dữ liệu lịch sử không thể được giữ lại đầy đủ. Tuy nhiên, hạn chế của nhóm này là hiệu quả phụ thuộc mạnh vào việc xác định đúng thành phần nào cần được bảo vệ và mức độ ràng buộc nên mạnh đến đâu. Nếu ràng buộc quá yếu, mô hình vẫn có thể quên tri thức cũ; ngược lại, nếu ràng buộc quá mạnh, mô hình sẽ khó thích nghi với dữ liệu mới.

Phương pháp dựa trên phát lại dữ liệu

Phương pháp dựa này đưa thông tin của các tác vụ cũ quay lại quá trình học tác vụ mới. Thông tin này có thể là dữ liệu gốc được lưu lại, một tập mẫu đại diện, hoặc dữ liệu được

sinh lại bởi một mô hình sinh. Nếu ký hiệu $\mathcal{B}_{<t}$ là bộ nhớ chứa thông tin đại diện từ các tác vụ trước, mục tiêu huấn luyện có thể được mô tả khái quát như sau:

$$\mathcal{L}(\theta) = \mathcal{L}_t(\theta; \mathcal{D}_t) + \beta \mathcal{L}_{\text{replay}}(\theta; \mathcal{B}_{<t}),$$

trong đó $\mathcal{D}_t$ là dữ liệu của tác vụ hiện tại, $\mathcal{B}_{<t}$ là bộ nhớ hoặc thông tin đại diện của các tác vụ cũ, và β điều khiển mức ảnh hưởng của thành phần phát lại.

Ưu điểm của replay là trực quan và thường hiệu quả, vì mô hình vẫn được tiếp xúc với phân phối cũ trong quá trình cập nhật. Tuy nhiên, hạn chế chính là chi phí lưu trữ có thể tăng theo thời gian khi số lượng tác vụ lớn. Ngoài ra, trong nhiều ứng dụng thực tế, việc lưu dữ liệu cũ có thể bị giới hạn bởi quyền riêng tư, dung lượng bộ nhớ hoặc yêu cầu vận hành. Nếu dùng mô hình sinh để thay thế dữ liệu gốc, hệ thống lại phải trả thêm chi phí huấn luyện và suy luận cho mô hình sinh.

Phương pháp dựa trên tối ưu hóa

Phương pháp này can thiệp trực tiếp vào quá trình cập nhật tham số. Thay vì chỉ thêm ràng buộc vào hàm mất mát hoặc phát lại dữ liệu cũ, nhóm phương pháp này điều chỉnh hướng gradient để giảm nguy cơ cập nhật mới gây hại cho các tác vụ trước. Một cách khái quát, nếu g là gradient trên dữ liệu hiện tại và g_k là gradient liên quan đến tác vụ cũ $\mathcal{T}_k$, quá trình cập nhật có thể đặt ra ràng buộc:

$$g^\top g_k \geq 0, \quad \forall k < t.$$

Ràng buộc này thể hiện ý tưởng rằng hướng cập nhật hiện tại không nên làm tăng mất mát trên các tác vụ đã học.

Ưu điểm của nhóm tối ưu hóa là kiểm soát trực tiếp tác động của cập nhật mới lên tri thức cũ, thay vì chỉ bảo vệ tham số một cách gián tiếp. Tuy nhiên, hạn chế là chi phí tính toán thường cao hơn do phải tính toán, kiểm tra hoặc điều chỉnh gradient theo các ràng buộc bổ sung. Khi số lượng tác vụ tăng, số lượng ràng buộc cũng có thể tăng, làm quá trình huấn luyện trở nên phức tạp hơn.

Phương pháp dựa trên biểu diễn

Phương pháp này tập trung vào việc xây dựng hoặc duy trì một không gian đặc trưng có khả năng khái quát tốt qua nhiều tác vụ. Thay vì chỉ bảo vệ từng tham số riêng lẻ, nhóm

phương pháp này hướng đến việc tạo ra các embedding ổn định hơn, trong đó dữ liệu từ nhiều tác vụ khác nhau vẫn có thể được biểu diễn một cách có cấu trúc. Theo Wang và cộng sự, hướng này bao gồm các chiến lược như tiền huấn luyện, học tự giám sát, học biểu diễn có khả năng chuyển giao và các cơ chế bảo toàn đặc trưng [4].

Ưu điểm của nhóm biểu diễn là có thể giúp mô hình giảm phụ thuộc vào dữ liệu cũ nếu không gian đặc trưng đủ ổn định và giàu khả năng khái quát. Trong các bài toán thị giác, điều này đặc biệt quan trọng vì chất lượng embedding ảnh hưởng trực tiếp đến các bước xử lý phía sau. Đối với phát hiện bất thường công nghiệp, hướng tiếp cận này có liên hệ chặt chẽ với việc sử dụng backbone đã huấn luyện trước, memory bank và coresets trong không gian đặc trưng. Tuy nhiên, hạn chế của nhóm này là hiệu quả phụ thuộc nhiều vào chất lượng biểu diễn ban đầu. Nếu backbone hoặc embedding không phản ánh tốt sự thay đổi của dữ liệu mới, mô hình vẫn có thể gặp khó khăn khi thích nghi với các tác vụ hoặc miền dữ liệu khác biệt.

Phương pháp dựa trên kiến trúc

Phương pháp này giảm quên thảm khốc bằng cách thay đổi hoặc mở rộng cấu trúc mô hình khi xuất hiện tác vụ mới. Thay vì buộc toàn bộ tác vụ chia sẻ cùng một tập tham số, mô hình có thể bổ sung nhánh, module hoặc thành phần chuyên biệt cho từng tác vụ. Cụ thể, sau khi học đến tác vụ thứ t , tham số của mô hình có thể được tổ chức thành:

$$\Theta_t = \Theta_{\text{shared}} \cup \Theta_1 \cup \Theta_2 \cup \dots \cup \Theta_t,$$

trong đó Θ_{shared} là phần tham số dùng chung, còn Θ_k là phần tham số chuyên biệt cho tác vụ $\mathcal{T}_k$.

Ưu điểm của nhóm kiến trúc là tri thức cũ có thể được bảo vệ tương đối trực tiếp bằng cách tách riêng hoặc đóng băng một phần tham số. Cách làm này giảm sự can thiệp giữa các tác vụ và giúp mô hình có thêm năng lực biểu diễn cho dữ liệu mới. Tuy nhiên, hạn chế lớn là số lượng tham số có thể tăng dần theo số lượng tác vụ. Điều này gây khó khăn cho triển khai lâu dài, đặc biệt trong các hệ thống có giới hạn về bộ nhớ, tốc độ suy luận hoặc tài nguyên tính toán.

Nhìn chung, năm nhóm phương pháp trên phản ánh các cách khác nhau để xử lý mâu thuẫn giữa học tri thức mới và bảo toàn tri thức cũ. Phương pháp điều chuẩn bảo vệ tham số hoặc hành vi quan trọng; phương pháp phát lại dữ liệu đưa thông tin cũ quay lại quá

trình học; phương pháp tối ưu hóa điều chỉnh trực tiếp hướng cập nhật; phương pháp biểu diễn hướng đến không gian đặc trưng bền vững hơn; còn phương pháp kiến trúc giảm sự can thiệp bằng cách phân tách hoặc mở rộng mô hình. Đây là cơ sở quan trọng để chuyển sang bài toán phát hiện bất thường liên tục, nơi mô hình không chỉ cần hạn chế quên thảm khốc mà còn phải duy trì khả năng biểu diễn dữ liệu bình thường trong điều kiện bộ nhớ và tài nguyên tính toán bị giới hạn.

2.4 Phát hiện bất thường liên tục

2.4.1 Tổng quan

Phát hiện bất thường liên tục (Continual Anomaly Detection – CAD), hay học liên tục cho phát hiện bất thường (Continual Learning for Anomaly Detection – CLAD), mở rộng bài toán phát hiện bất thường sang thiết lập dữ liệu xuất hiện tuần tự theo thời gian. Thay vì xây dựng một mô hình trên tập ảnh bình thường cố định, CAD giả định hệ thống lần lượt tiếp nhận các nhóm sản phẩm, miền dữ liệu hoặc điều kiện vận hành mới. Sau mỗi lần cập nhật, mô hình cần nhận biết bất thường trên dữ liệu mới nhưng không được suy giảm mạnh trên các nhóm dữ liệu đã học [14, 8, 15].

Trong kiểm tra lỗi công nghiệp, CAD phản ánh rõ hơn quá trình vận hành thực tế. Một dây chuyền có thể bổ sung sản phẩm mới, thay đổi camera, thay đổi ánh sáng hoặc phát sinh các trạng thái bình thường mới theo thời gian. Nếu mô hình chỉ được huấn luyện một lần, vùng biểu diễn bình thường ban đầu có thể không còn phù hợp với dữ liệu mới. Ngược lại, nếu cập nhật trực tiếp theo dữ liệu mới mà không kiểm soát, các đại diện bình thường của sản phẩm cũ có thể bị thay thế, làm tăng cảnh báo sai khi gặp lại dữ liệu lịch sử.

Với bài toán định vị bất thường ở mức điểm ảnh, yêu cầu này còn khó hơn. Mô hình không chỉ cần phân biệt ảnh bình thường và ảnh lỗi, mà còn phải duy trì bản đồ bất thường ổn định qua nhiều tác vụ. Dalle Pezze và cộng sự đặt bài toán pixel-level anomaly detection trong thiết lập học liên tục, trong đó mỗi tác vụ tương ứng với một nhóm sản phẩm và mô hình được đánh giá trên cả sản phẩm mới lẫn các sản phẩm đã học [14]. Các benchmark gần đây về continual anomaly localization cũng nhấn mạnh rằng hiệu năng cần được đánh giá đồng thời ở cấp ảnh và cấp điểm ảnh [9].

Từ đó, CAD có ba yêu cầu chính: cập nhật được biểu diễn bình thường cho tác vụ mới, giữ lại các đại diện quan trọng của tác vụ cũ, và kiểm soát chi phí bộ nhớ khi số lượng tác

vụ tăng. Từ các yêu cầu trên, những phương pháp giải quyết bài toán phát hiện bất thường liên tục có thể được nhìn nhận theo một số hướng chính: phát lại dữ liệu lịch sử, sử dụng prompt hoặc bộ nhớ theo tác vụ, và cập nhật bộ nhớ đặc trưng bằng coreset.

2.4.2 Các hướng tiếp cận giải quyết bài toán phát hiện bất thường liên tục

Hướng tiếp cận dựa trên phát lại dữ liệu

Nhóm phương pháp phát lại dữ liệu (replay) giảm quên thảm khốc bằng cách đưa thông tin của các tác vụ cũ quay lại khi mô hình học tác vụ mới. Trong CAD, thông tin được phát lại có thể là ảnh gốc, ảnh đã nén, đặc trưng trung gian hoặc ảnh được sinh lại bởi mô hình sinh. Với bài toán định vị bất thường, replay không chỉ cần giữ nhãn ảnh hoặc đặc trưng toàn cục, mà còn phải bảo toàn cấu trúc không gian của vùng ảnh, vì điểm bất thường thường được tính tại mức patch hoặc mức điểm ảnh.

Dalle Pezze và cộng sự nghiên cứu pixel-level anomaly detection trong thiết lập học liên tục và so sánh ba dạng replay: lưu ảnh trực tiếp, compressed replay và generative replay [14]. Với compressed replay, phương pháp SCALE lưu phiên bản nén của ảnh lịch sử, sau đó dùng super-resolution để phục hồi ảnh khi phát lại. Cách này làm giảm dung lượng lưu trữ so với lưu toàn bộ ảnh gốc, nhưng chất lượng bản đồ bất thường phụ thuộc vào mức độ bảo toàn chi tiết sau quá trình nén và phục hồi. Nếu các chi tiết bề mặt nhỏ bị mất, mô hình có thể khó duy trì khả năng định vị lỗi trên tác vụ cũ.

ReplayCAD đi theo hướng generative replay. Thay vì lưu trực tiếp ảnh lịch sử, phương pháp sử dụng mô hình khuếch tán đã huấn luyện trước để sinh lại dữ liệu của các tác vụ cũ [16]. Quá trình sinh được điều kiện hóa bằng thông tin ngữ nghĩa của lớp sản phẩm và đặc trưng không gian, nhằm tạo ra ảnh phát lại có cấu trúc phù hợp với từng nhóm dữ liệu. Nhờ đó, mô hình có thể học tác vụ mới trong khi vẫn tiếp xúc với các mẫu đại diện cho phân phối cũ, đặc biệt hữu ích đối với anomaly localization vì cấu trúc hình ảnh được đưa trở lại quá trình cập nhật.

Ưu điểm của replay là cơ chế chống quên khá trực tiếp. Thay vì chỉ ràng buộc tham số hoặc lưu thống kê, mô hình được huấn luyện lại trên dữ liệu đại diện cho các tác vụ trước đó. Tuy nhiên, hướng này đánh đổi bằng chi phí tài nguyên. Lưu ảnh thật làm tăng dung lượng bộ nhớ; lưu ảnh nén phụ thuộc vào chất lượng phục hồi; còn generative replay cần thêm mô hình sinh, làm tăng chi phí huấn luyện, lưu trữ và suy luận. Ngoài ra, nếu dữ liệu phát lại không đủ đa dạng hoặc không giữ được các chi tiết lỗi nhỏ ở mức bề mặt, hiệu

qua duy trì khả năng định vị bất thường trên tác vụ cũ có thể bị hạn chế.

Hướng tiếp cận dựa trên prompt và bộ nhớ theo tác vụ

Các phương pháp dựa trên prompt tiếp cận CAD theo hướng giảm mức độ can thiệp vào mô hình nền. Thay vì tinh chỉnh toàn bộ backbone khi xuất hiện tác vụ mới, hệ thống giữ phần trích xuất đặc trưng tương đối ổn định và bổ sung các thành phần điều khiển như key, prompt hoặc knowledge memory. Các thành phần này đóng vai trò mã hóa thông tin ngữ cảnh của từng tác vụ, giúp mô hình chọn đúng tri thức bình thường cần dùng trong quá trình suy luận.

UCAD còn sử dụng Structure-based Contrastive Learning với Segment Anything Model. Các mặt nạ cấu trúc do SAM sinh ra được dùng để tạo ràng buộc ở mức vùng: đặc trưng trong cùng một vùng cấu trúc được kéo gần nhau, còn đặc trưng thuộc các vùng khác nhau được đẩy xa nhau [17]. Cách làm này nhắm trực tiếp vào anomaly segmentation, vì lỗi công nghiệp thường chỉ xuất hiện ở những vùng nhỏ và cần biểu diễn patch đủ phân biệt.

Về mặt biểu diễn, UCAD không chỉ dựa vào prompt được học trực tiếp từ dữ liệu bình thường. Phương pháp này còn sử dụng Structure-based Contrastive Learning (SCL) với Segment Anything Model (SAM). Các mặt nạ cấu trúc do SAM tạo ra được dùng như thông tin phân vùng: đặc trưng thuộc cùng một vùng cấu trúc được kéo lại gần nhau, trong khi đặc trưng thuộc các vùng khác nhau được đẩy ra xa nhau [17]. Cơ chế này giúp đặc trưng patch trở nên gọn và phân biệt hơn, điều có ý nghĩa trực tiếp đối với anomaly segmentation, nơi mô hình phải phân biệt các sai lệch nhỏ ở mức cục bộ.

Ưu điểm của hướng prompt là khả năng thích nghi với tác vụ mới mà không cần cập nhật toàn bộ backbone. Khi backbone lớn hoặc đã được pretrain tốt, việc học thêm prompt và knowledge thường nhẹ hơn so với tinh chỉnh toàn bộ mô hình. Cách tổ chức key–prompt–knowledge cũng giúp mô hình lưu tri thức theo dạng có cấu trúc: key phục vụ chọn tác vụ, prompt phục vụ điều chỉnh biểu diễn, còn knowledge phục vụ so khớp bất thường.

Hạn chế của hướng này nằm ở cơ chế truy xuất và tổ chức bộ nhớ. Nếu key không phân biệt tốt giữa các tác vụ gần nhau, hệ thống có thể truy xuất sai prompt hoặc sai knowledge, làm tăng cảnh báo sai hoặc làm giảm khả năng định vị lỗi. Ngoài ra, khi số lượng tác vụ tăng, bộ nhớ prompt/knowledge có thể bị phân mảnh theo tác vụ. Mỗi tác vụ có thể cần một tập key, prompt và knowledge riêng, khiến hệ thống khó duy trì một không gian biểu diễn thống nhất nếu không có cơ chế nén hoặc hợp nhất bộ nhớ hiệu quả.

Hướng tiếp cận dựa trên coreset và ngân hàng bộ nhớ hợp nhất

Các phương pháp dựa trên embedding thường lưu đặc trưng bình thường trong memory bank và tính điểm bất thường bằng khoảng cách lân cận gần nhất. Trong thiết lập tĩnh, memory bank được xây dựng một lần từ tập ảnh bình thường. Khi chuyển sang CAD, bộ nhớ này phải được cập nhật sau mỗi tác vụ mới. Nếu cộng dồn toàn bộ embedding mới, dung lượng lưu trữ và chi phí tìm kiếm sẽ tăng dần theo số tác vụ. Nếu tách memory bank riêng cho từng tác vụ, hệ thống lại cần biết task identity khi suy luận và dễ gặp tình trạng phân mảnh tri thức.

CADIC xử lý vấn đề này bằng cách tổ chức các tác vụ trong một ngân hàng bộ nhớ hợp nhất (unified memory bank), thay vì duy trì các sub-memory bank riêng cho từng tác vụ. Phương pháp hướng đến việc học các mẫu bình thường của tác vụ mới dưới giả định phân phối dữ liệu thay đổi, đồng thời giảm quên thảm khốc trên những tác vụ đã học [15]. Trong quá trình huấn luyện, bộ nhớ được cập nhật tăng dần bằng fixed-size incremental coreset. Nói cách khác, hệ thống chỉ giữ lại một tập embedding đại diện trong giới hạn bộ nhớ cố định, thay vì lưu toàn bộ embedding sinh ra qua các tác vụ.

Cập nhật bằng coreset giúp memory bank giữ được các mẫu đặc trưng tiêu biểu của nhiều tác vụ trong cùng một không gian. Khi tác vụ mới xuất hiện, các embedding mới không được thêm vào bộ nhớ một cách không kiểm soát, mà phải trải qua bước chọn lọc để duy trì kích thước bộ nhớ. Điều này làm cho CADIC gần với tinh thần của PatchCore ở chỗ dùng memory bank và nearest-neighbor scoring, nhưng khác ở chỗ memory bank không còn là cấu trúc tĩnh. Bộ nhớ được cập nhật liên tục theo chuỗi tác vụ và phải giữ cân bằng giữa embedding cũ với embedding mới.

Ở giai đoạn suy luận, CADIC tính điểm bất thường bằng nearest-neighbor matching giữa embedding của ảnh kiểm thử và các embedding bình thường trong unified memory bank [15]. Nếu một patch kiểm thử nằm xa các embedding bình thường đã lưu, patch đó sẽ nhận điểm bất thường cao hơn. Do toàn bộ tác vụ cùng chia sẻ một memory bank, mô hình không cần chọn bộ nhớ riêng cho từng tác vụ trước khi suy luận, phù hợp với bài toán CAD, nơi task identity không phải lúc nào cũng có sẵn ở thời điểm kiểm thử.

Ưu điểm chính của hướng coreset và bộ nhớ hợp nhất là khả năng kiểm soát chi phí lưu trữ. Bộ nhớ không tăng tuyến tính theo số tác vụ, trong khi hệ thống vẫn giữ lại các embedding đại diện cho phân phối bình thường đã gặp. Cách làm này phù hợp với các hệ thống công nghiệp cần cập nhật theo thời gian nhưng bị giới hạn bởi bộ nhớ và thời gian

suy luận. Ngoài ra, vì scoring vẫn dựa trên embedding và nearest neighbor, hướng này có thể tận dụng trực tiếp các backbone thị giác mạnh đã được tiền huấn luyện.

Hạn chế của hướng này nằm ở chất lượng chọn mẫu đại diện. Nếu coreset không bao phủ tốt phân phối của tác vụ cũ, một số vùng bình thường quan trọng có thể bị loại khỏi bộ nhớ, làm tăng nguy cơ cảnh báo sai khi gặp lại dữ liệu cũ. Ngược lại, nếu coreset ưu tiên giữ quá nhiều embedding cũ, bộ nhớ có thể thiếu khả năng đại diện cho tác vụ mới. Do đó, hiệu quả của CADIC phụ thuộc mạnh vào chiến lược cập nhật coreset, chất lượng embedding đầu vào và cách cân bằng giữa dữ liệu cũ với dữ liệu mới.

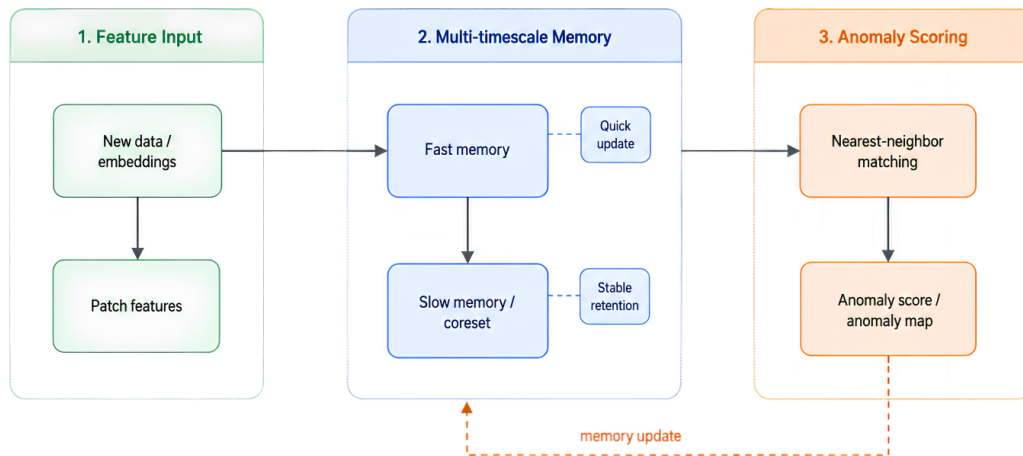
2.5 Học lồng nhau

2.5.1 Khái niệm Học lồng nhau

Học lồng nhau (Nested Learning – NL) là một hướng tiếp cận gần đây nhằm mô tả mô hình học máy như một hệ gồm nhiều quá trình học được tổ chức ở các tầng khác nhau. Thay vì chỉ xem mô hình như một kiến trúc nơ-ron cố định được huấn luyện trên một tập dữ liệu, NL xem cả kiến trúc, bộ nhớ và quá trình tối ưu như các thành phần cùng tham gia vào một hệ thống học nhiều tầng [18]. Trong hệ thống đó, mỗi tầng có ngữ cảnh xử lý riêng, trạng thái riêng và tần suất cập nhật riêng.

Trong các mô hình học sâu thông thường, học thường được hiểu là quá trình cập nhật tham số trong giai đoạn huấn luyện. Sau khi huấn luyện xong, phần lớn tham số được giữ cố định và mô hình chỉ thực hiện suy luận trên dữ liệu đầu vào. Thay vào đó, Học lồng nhau mở rộng hơn về khái niệm "học". Ngoài tham số chính của mạng, các trạng thái như bộ nhớ, momentum của bộ tối ưu, trạng thái ẩn (hidden states) hoặc các module thích nghi cũng có thể được xem là nơi lưu giữ thông tin. Vì vậy, khả năng học của mô hình còn phụ thuộc vào cách các tầng trạng thái này được cập nhật và truyền thông tin cho nhau.

Cách diễn giải này có ý nghĩa đối với các bài toán học liên tục. Khi dữ liệu thay đổi theo thời gian, một mô hình hoàn toàn tĩnh sẽ khó thích nghi với phân phối mới. Ngược lại, nếu mọi thành phần đều được cập nhật trực tiếp theo dữ liệu mới, mô hình dễ làm suy giảm tri thức đã học. Học lồng nhau gợi ý rằng quá trình học nên được phân tách thành nhiều tầng: có tầng phản ứng nhanh với dữ liệu mới, có tầng cập nhật chậm hơn để giữ tri thức ổn định, và có cơ chế trung gian để quyết định thông tin nào cần được củng cố.



Hình 5: Sơ đồ bộ nhớ nhiều thang thời gian cho phát hiện bất thường liên tục. Tầng cập nhật nhanh hỗ trợ thích nghi với đặc trưng mới, trong khi tầng cập nhật chậm duy trì các đại diện ổn định dùng cho so khớp lân cận gần nhất.

2.5.2 Mô hình như một hệ tối ưu nhiều tầng

Trong Học lồng nhau, một mô hình học máy được nhìn như một hệ gồm nhiều bài toán tối ưu lồng nhau, đa tầng hoặc song song. Mỗi tầng có ngữ cảnh xử lý riêng: tầng huấn luyện xử lý dữ liệu huấn luyện, tầng tối ưu xử lý chuỗi gradient, còn tầng bộ nhớ có thể xử lý token, embedding, đặc trưng ảnh hoặc trạng thái trung gian của mô hình [18]. Cách nhìn này làm mờ ranh giới giữa kiến trúc, bộ nhớ và thuật toán tối ưu, vì các thành phần này đều có thể được xem như những cơ chế lưu trữ và cập nhật thông tin.

Từ góc nhìn đó, một lớp mạng được cập nhật bằng lan truyền ngược có thể được hiểu như một bộ nhớ lưu quan hệ giữa đầu vào và tín hiệu sai lệch cục bộ. Tương tự, momentum trong bộ tối ưu không chỉ là kỹ thuật làm mượt gradient, mà còn là trạng thái nén thông tin cập nhật trong quá khứ. Vì vậy, quá trình học liên tục không nên chỉ được phân tích ở mức tham số cuối cùng của mô hình, mà cần xét cả các trạng thái trung gian như gradient, bộ nhớ và embedding đại diện.

Vấn đề này còn liên hệ trực tiếp với vấn đề quên thảm khốc. Khi mô hình học tác vụ mới, nếu các trạng thái mang thông tin lịch sử không được bảo toàn hoặc chuyển tiếp hợp lý, cập nhật mới có thể làm suy giảm tri thức đã học. Một số hướng gần với quan điểm này có thể thấy trong các mô hình học tại thời điểm kiểm thử, chẳng hạn như Test-Time Training cập nhật hidden state của RNN bằng mục tiêu tự giám sát (self – supervised) trên chuỗi đầu vào [19], trong khi Titans sử dụng neural memory để lưu thông tin dài hạn bên cạnh attention cho ngữ cảnh gần [20]. Các hướng này không trùng hoàn toàn với Học lồng nhau, nhưng cùng cho thấy xu hướng xem trạng thái nội tại của mô hình như một thành

phần có khả năng học.

2.5.3 Bộ nhớ đa thang thời gian (multi-timescale)

Học lồng nhau tổ chức bộ nhớ theo nhiều thang thời gian. Thay vì phân chia bộ nhớ thành hai nhóm rời rạc như ngắn hạn và dài hạn, NL đề xuất Continuum Memory System (CMS), trong đó các tầng bộ nhớ có tần suất cập nhật khác nhau [18]. Các tầng cập nhật nhanh có thể ghi nhận những biến đổi vừa xuất hiện trong dữ liệu, nhưng cũng dễ bị ảnh hưởng bởi nhiễu hoặc các thay đổi ngắn hạn. Ngược lại, các tầng cập nhật chậm hơn ít phản ứng với từng mẫu mới, nhưng phù hợp để giữ các đặc trưng đã xuất hiện ổn định qua nhiều giai đoạn học.

Khác với memory bank thông thường, nơi bộ nhớ chủ yếu được xem như một tập vector dùng cho so khớp lân cận, bộ nhớ đa thang thời gian đặt trọng tâm vào quy tắc cập nhật của bộ nhớ. Vấn đề không chỉ là lưu những đặc trưng nào, mà phải phân biệt đặc trưng nào được cập nhật ngay, đặc trưng nào cần được củng cố qua thời gian và đặc trưng nào có thể bị thay thế.

Các thực nghiệm trong công trình gốc của học lồng nhau chủ yếu tập trung vào mô hình ngôn ngữ và chuỗi dài. Trong đề tài này, nhóm đồ án không sử dụng trực tiếp kiến trúc đó, mà khai thác ý tưởng tổ chức bộ nhớ nhiều tầng và cập nhật theo nhiều thang thời gian cho bài toán phát hiện bất thường liên tục.

2.6 Mô hình thị giác nền tảng cho trích xuất đặc trưng ảnh

Các mô hình thị giác nền tảng được huấn luyện trên dữ liệu ảnh quy mô lớn nhằm học biểu diễn thị giác có khả năng chuyển giao sang nhiều tác vụ khác nhau. Trong các bài toán phát hiện bất thường dựa trên đặc trưng, các mô hình này thường được dùng như backbone trích xuất embedding, sau đó kết hợp với memory bank, coreset hoặc cơ chế so khớp lân cận gần nhất.

DINOv2 là mô hình thị giác nền tảng do Meta AI Research/FAIR phát triển, được huấn luyện theo hướng tự giám sát trên một tập dữ liệu ảnh lớn đã qua chọn lọc [21]. Mô hình sử dụng kiến trúc Vision Transformer và được thiết kế để tạo ra đặc trưng thị giác tổng quát mà không cần phụ thuộc vào nhãn thủ công. Công trình gốc cũng nhấn mạnh rằng đặc trưng của DINOv2 có thể được sử dụng hiệu quả cho nhiều benchmark ở cả cấp ảnh và cấp điểm ảnh mà không cần fine-tuning cho từng tác vụ cụ thể [21].

DINOv2 phù hợp với phát hiện bất thường ở khả năng tạo biểu diễn ở mức patch. Bên cạnh vector biểu diễn toàn cục của ảnh, mô hình còn cung cấp các embedding cục bộ tương ứng với những vùng không gian khác nhau. Nhờ đó, đặc trưng DINOv2 có thể được dùng không chỉ cho phát hiện bất thường ở cấp ảnh, mà còn cho các bài toán cần định vị vùng lỗi ở cấp điểm ảnh.

Trong phạm vi đồ án, DINOv2 được sử dụng như bộ trích xuất đặc trưng ảnh đã huấn luyện trước. Các embedding của ảnh bình thường được đưa vào cơ chế bộ nhớ đặc trưng, còn ảnh kiểm thử được đánh giá thông qua khoảng cách trong không gian embedding. Cách sử dụng này phù hợp với hướng tiếp cận của đề tài, vì trọng tâm không nằm ở việc huấn luyện lại backbone thị giác, mà ở cách tổ chức và cập nhật bộ nhớ đặc trưng trong thiết lập phát hiện bất thường liên tục.

Chương 3

Cơ sở lý thuyết

3.1 Biểu diễn đặc trưng trong phát hiện bất thường công nghiệp

3.1.1 Biểu diễn ảnh bằng patch embedding

Trong đề tài này, ảnh đầu vào được xử lý bằng một bộ trích xuất đặc trưng thị giác cố định. Bộ trích xuất này không trực tiếp dự đoán nhãn bình thường hoặc bất thường, mà ánh xạ ảnh sang không gian embedding để phục vụ bước so khớp với các đặc trưng bình thường đã lưu. Backbone được giữ cố định nhằm tách phần biểu diễn ảnh khỏi phần cập nhật bộ nhớ liên tục.

Gọi $x \in \mathbb{R}^{H \times W \times 3}$ là ảnh đầu vào và $\phi(\cdot)$ là bộ trích xuất đặc trưng. Khi đưa ảnh qua ϕ , ta thu được embedding mức ảnh và embedding mức patch:

$$\phi(x) = (z_{\text{cls}}, Z_{\text{patch}}),$$

trong đó:

$$z_{\text{cls}} \in \mathbb{R}^d, \quad Z_{\text{patch}} = \{\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_N\}, \quad \mathbf{z}_i \in \mathbb{R}^d.$$

Ở đây, z_{cls} biểu diễn đặc trưng tổng quát của toàn ảnh, còn $\mathbf{z}_i$ là đặc trưng của patch thứ i . Việc sử dụng embedding mức patch phù hợp với phát hiện bất thường công nghiệp vì nhiều lỗi chỉ xuất hiện ở một vùng nhỏ trên ảnh. Do đó, hệ thống cần giữ lại các đặc trưng cục bộ để phục vụ định vị vùng bất thường, thay vì chỉ dùng một vector đại diện cho toàn ảnh.

Trong pipeline của đề tài, embedding mức ảnh được dùng cho các thành phần cần biểu diễn tổng quát của ảnh, còn embedding mức patch được dùng để tính điểm bất thường cục bộ và tạo bản đồ bất thường. Với hướng tiếp cận dựa trên memory bank, ảnh kiểm thử được đánh giá bằng cách so khớp embedding của nó với các embedding bình thường đã lưu, thay vì phân loại trực tiếp bằng một classifier [3, 15].

3.1.2 Điểm bất thường trong không gian đặc trưng

Trong phần tính điểm bất thường cục bộ, $\mathcal{C}_{\text{patch}}$ được hiểu là coreset chứa các patch embedding bình thường. Gọi:

$$\mathcal{C}_{\text{patch}} = \{\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_M\}, \quad \mathbf{c}_j \in \mathbb{R}^d.$$

Với một patch embedding $\mathbf{z}_i$ của ảnh kiểm thử, điểm bất thường cục bộ được tính bằng khoảng cách đến embedding bình thường gần nhất trong $\mathcal{C}_{\text{patch}}$:

$$s_i = \min_{\mathbf{c} \in \mathcal{C}_{\text{patch}}} \|\mathbf{z}_i - \mathbf{c}\|_2.$$

Cách tính này xuất phát từ các phương pháp memory bank như PatchCore, trong đó một patch được xem là bất thường nếu đặc trưng của nó nằm xa các đặc trưng bình thường đã lưu [3]. Trong thiết lập phát hiện bất thường liên tục, CADIC kế thừa ý tưởng so khớp lân cận gần nhất nhưng thay memory bank tĩnh bằng incremental coreset để bộ nhớ có thể được cập nhật qua nhiều tác vụ [15].

Tập các điểm $\{s_i\}_{i=1}^N$ tạo thành bản đồ điểm bất thường ở mức patch. Bản đồ này sau đó được nội suy về kích thước ảnh gốc để phục vụ đánh giá và trực quan hóa ở mức điểm ảnh. Gọi điểm patch lớn nhất là:

$$s^* = \max_i s_i.$$

Gọi $i^* = \arg \max_i s_i$ là vị trí patch có điểm bất thường lớn nhất, $\mathbf{z}^* = \mathbf{z}_{i^*}$ là embedding tương ứng, và $\mathbf{c}^*$ là embedding bình thường gần nhất với $\mathbf{z}^*$:

$$\mathbf{c}^* = \arg \min_{\mathbf{c} \in \mathcal{C}_{\text{patch}}} \|\mathbf{z}^* - \mathbf{c}\|_2.$$

Theo cách hiệu chỉnh điểm ảnh trong PatchCore và được CADIC sử dụng lại trong thiết lập coreset tăng dần, điểm bất thường cấp ảnh có thể được tính bằng trọng số lân cận quanh $\mathbf{c}^*$ [3, 15]. Gọi $N_b(\mathbf{c}^*)$ là tập b lân cận gần nhất của $\mathbf{c}^*$ trong coreset $\mathcal{C}_{\text{patch}}$, điểm bất thường cấp ảnh được viết là:

$$s_{\text{img}} = \left(1 - \frac{\exp(\|\mathbf{z}^* - \mathbf{c}^*\|_2)}{\sum_{\mathbf{c} \in N_b(\mathbf{c}^*)} \exp(\|\mathbf{z}^* - \mathbf{c}\|_2)} \right) s^*.$$

Trong công thức trên, s^* phản ánh mức bất thường lớn nhất trong ảnh, còn thành phần trong ngoặc đóng vai trò hiệu chỉnh theo mật độ lân cận của embedding bình thường gần nhất. Trong đề tài này, s_{img} được dùng cho đánh giá cấp ảnh, còn tập điểm $\{s_i\}_{i=1}^N$ được dùng để xây dựng bản đồ bất thường ở cấp patch và cấp điểm ảnh.

3.2 Bộ nhớ đặc trưng và coreset trong phát hiện bất thường liên tục

3.2.1 Ngân hàng bộ nhớ đặc trưng

Trong bài toán phát hiện bất thường liên tục, hệ thống cần cập nhật tri thức về dữ liệu bình thường khi xuất hiện tác vụ mới mà không huấn luyện lại toàn bộ mô hình từ đầu. Với hướng tiếp cận dựa trên đặc trưng, tri thức này được biểu diễn bằng một ngân hàng bộ nhớ chứa các embedding của mẫu bình thường. Ở giai đoạn suy luận, embedding của ảnh kiểm thử được so khớp với các embedding bình thường trong bộ nhớ; khoảng cách lớn đến bộ nhớ cho thấy mẫu kiểm thử có khả năng bất thường cao.

Trong đề án này, ngân hàng bộ nhớ chính được ký hiệu là $\mathcal{C}_{\text{patch}}$. Mỗi phần tử $\mathbf{c} \in \mathcal{C}_{\text{patch}}$ là một patch embedding đại diện cho dữ liệu bình thường đã được quan sát. Khi một batch embedding mới $\mathcal{X}_t$ được trích xuất từ dữ liệu bình thường của tác vụ hiện tại, bộ nhớ được cập nhật theo cơ chế incremental coreset của CADIC [15]. Mục tiêu của bước cập nhật là bổ sung các embedding mới chưa được bộ nhớ hiện tại bao phủ tốt, đồng thời loại bỏ các embedding dư thừa trong coreset.

3.2.2 Cập nhật incremental coreset

Với mỗi embedding mới $\mathbf{x} \in \mathcal{X}_t$, CADIC đo khoảng cách từ $\mathbf{x}$ đến phần tử gần nhất trong coreset hiện tại $\mathcal{C}_{\text{patch}}$ [15]:

$$d(\mathbf{x}, \mathcal{C}_{\text{patch}}) = \min_{\mathbf{c} \in \mathcal{C}_{\text{patch}}} \|\mathbf{x} - \mathbf{c}\|_2.$$

Giá trị này cho biết embedding mới đã được bộ nhớ hiện tại đại diện tốt hay chưa. Nếu $d(\mathbf{x}, \mathcal{C}_{\text{patch}})$ lớn, $\mathbf{x}$ nằm xa các mẫu bình thường đã lưu và có thể mang thông tin mới cần được đưa vào bộ nhớ.

Trong batch embedding mới $\mathcal{X}_t$, embedding xa coreset nhất được xác định bởi [15]:

$$d_{\max}(\mathcal{X}_t, \mathcal{C}_{\text{patch}}) = \max_{\mathbf{x} \in \mathcal{X}_t} d(\mathbf{x}, \mathcal{C}_{\text{patch}}),$$

$$\mathbf{x}^* = \arg \max_{\mathbf{x} \in \mathcal{X}_t} d(\mathbf{x}, \mathcal{C}_{\text{patch}}).$$

Để xác định mức dư thừa trong coreset hiện tại, ta xét khoảng cách nhỏ nhất giữa hai phần tử khác nhau trong $\mathcal{C}_{\text{patch}}$:

$$d_{\min}(\mathcal{C}_{\text{patch}}) = \min_{\substack{\mathbf{c}_1, \mathbf{c}_2 \in \mathcal{C}_{\text{patch}} \\ \mathbf{c}_1 \neq \mathbf{c}_2}} \|\mathbf{c}_1 - \mathbf{c}_2\|_2.$$

Cặp embedding gần nhau nhất trong coreset được xác định bởi:

$$\mathbf{c}_1^*, \mathbf{c}_2^* = \arg \min_{\substack{\mathbf{c}_1, \mathbf{c}_2 \in \mathcal{C}_{\text{patch}} \\ \mathbf{c}_1 \neq \mathbf{c}_2}} \|\mathbf{c}_1 - \mathbf{c}_2\|_2.$$

Bộ nhớ chỉ được cập nhật khi embedding mới xa coreset hiện tại hơn khoảng cách gần nhất giữa hai phần tử đang có trong coreset [15]:

$$d_{\max}(\mathcal{X}_t, \mathcal{C}_{\text{patch}}) > d_{\min}(\mathcal{C}_{\text{patch}}).$$

Khi điều kiện trên thỏa mãn, $\mathbf{x}^*$ được thêm vào $\mathcal{C}_{\text{patch}}$ và một phần tử trong cặp gần nhau nhất, chẳng hạn $\mathbf{c}_1^*$, được loại khỏi coreset:

$$\mathcal{C}_{\text{patch}} \leftarrow (\mathcal{C}_{\text{patch}} \cup \{\mathbf{x}^*\}) \setminus \{\mathbf{c}_1^*\}.$$

Nhờ đó, kích thước coreset được giữ cố định, trong khi bộ nhớ vẫn có thể tiếp nhận các đặc trưng bình thường mới qua từng tác vụ.

3.3 Thiết lập học liên tục cho phát hiện bất thường

3.3.1 Chuỗi tác vụ và dữ liệu bình thường

Giả sử hệ thống lần lượt tiếp nhận T tác vụ phát hiện bất thường:

$$\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_T.$$

Mỗi tác vụ $\mathcal{T}_t$ có tập huấn luyện $\mathcal{D}_t^{train}$ chủ yếu gồm các ảnh bình thường và tập kiểm thử $\mathcal{D}_t^{test}$ gồm cả ảnh bình thường lẫn ảnh bất thường. Sau khi học đến tác vụ thứ t , mô hình không chỉ được đánh giá trên tác vụ hiện tại $\mathcal{T}_t$, mà còn trên toàn bộ các tác vụ đã gặp:

$$\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_t.$$

Điểm khác biệt so với phát hiện bất thường tĩnh là bộ nhớ đặc trưng không còn được xây dựng một lần trên một tập dữ liệu cố định. Thay vào đó, bộ nhớ phải được cập nhật qua từng tác vụ sao cho có thể bao phủ dữ liệu bình thường mới, đồng thời hạn chế làm mất các đại diện quan trọng của dữ liệu bình thường cũ.

3.3.2 Cập nhật bộ nhớ qua nhiều tác vụ

Gọi $\mathcal{C}_{patch}^{(t)}$ là coreset sau khi hệ thống đã học xong tác vụ $\mathcal{T}_t$. Khi nhận dữ liệu bình thường của tác vụ mới, hệ thống trích xuất tập patch embedding $\mathcal{X}_t$ và cập nhật bộ nhớ theo dạng:

$$\mathcal{C}_{patch}^{(t)} = \text{Update} \left(\mathcal{C}_{patch}^{(t-1)}, \mathcal{X}_t \right).$$

Trong đồ án này, hàm $\text{Update}(\cdot)$ được triển khai bằng incremental coreset như đã trình bày ở trên. Cách cập nhật này giúp bộ nhớ không tăng không kiểm soát theo số lượng tác vụ, vì kích thước coreset luôn được giới hạn bởi một ngưỡng cố định M :

$$\left| \mathcal{C}_{patch}^{(t)} \right| \leq M.$$

Sau khi cập nhật đến tác vụ t , coreset $\mathcal{C}_{patch}^{(t)}$ được dùng để tính điểm bất thường cho các ảnh kiểm thử thuộc tất cả tác vụ đã gặp. Do đó, chất lượng của coreset ảnh hưởng trực

tiếp đến cả khả năng thích nghi với tác vụ mới và khả năng duy trì hiệu năng trên các tác vụ cũ.

3.4 Cơ sở học lồng nhau cho mô hình đề xuất

3.4.1 Hệ học nhiều tầng

Dựa trên góc nhìn của học lồng nhau, mô hình đề xuất được tổ chức thành các tầng xử lý có tốc độ cập nhật khác nhau [18]. Ở tầng thấp nhất, backbone thị giác tạo embedding từ ảnh đầu vào và được giữ cố định trong quá trình học liên tục. Ở các tầng phía sau, bộ nhớ nhanh và bộ nhớ chậm lần lượt xử lý việc thích nghi với đặc trưng mới và lưu giữ các đại diện ổn định của dữ liệu bình thường.

Một cách khái quát, đầu vào ảnh x_t được ánh xạ sang đặc trưng bởi bộ trích xuất đặc trưng cố định:

$$\mathbf{z}_t = \phi(x_t),$$

trong đó $\phi(\cdot)$ là backbone thị giác và $\mathbf{z}_t$ là embedding hoặc patch embedding của ảnh. Thay vì cập nhật trực tiếp $\phi(\cdot)$ khi có dữ liệu mới, mô hình chỉ cập nhật các trạng thái bộ nhớ phía sau:

$$\mathbf{h}_t = M_t^{fast}(\mathbf{z}_t), \quad \mathcal{C}_{patch}^{(t)} = M_t^{slow}(\mathcal{C}_{patch}^{(t-1)}, \mathbf{h}_t).$$

Trong đó, M_t^{fast} là bộ nhớ nhanh dùng để điều chỉnh biểu diễn hiện tại, còn M_t^{slow} là cơ chế cập nhật bộ nhớ chậm, được hiện thực bằng incremental coreset.

Với cách tổ chức trên, phần học liên tục của mô hình không nằm ở việc huấn luyện lại toàn bộ mạng. Thay vào đó, quá trình cập nhật được đưa vào các tầng bộ nhớ. Backbone giữ vai trò tạo biểu diễn ổn định, Fast Memory tiếp nhận thay đổi gần đây, còn Slow Memory duy trì tập đại diện dùng cho so khớp lân cận khi tính điểm bất thường.

3.4.2 Bộ nhớ cập nhật theo nhiều thang thời gian

Nested Learning đề xuất Continuum Memory System, trong đó các tầng bộ nhớ có tần suất cập nhật khác nhau [18]. Với mô hình đề xuất, ý tưởng này được dùng ở mức hệ

thông: Fast Memory cập nhật thường xuyên theo embedding mới, còn Slow Memory được cập nhật có chọn lọc để giữ kích thước bộ nhớ trong giới hạn.

Đối với Fast Memory, đề tài sử dụng dạng cập nhật tuyến tính lấy cảm hứng từ cơ chế neural memory của TITANS [20]. Với embedding đầu vào $\mathbf{z}_t$, khóa và giá trị được tạo bởi:

$$\mathbf{k}_t = W_K \mathbf{z}_t, \quad \mathbf{v}_t = W_V \mathbf{z}_t.$$

Bộ nhớ nhanh được cập nhật theo dạng Delta Rule:

$$M_t^{fast} = (1 - \alpha) M_{t-1}^{fast} + \eta_t \left(\mathbf{v}_t - M_{t-1}^{fast} \mathbf{k}_t \right) \mathbf{k}_t^\top.$$

Thành phần:

$$\mathbf{v}_t - M_{t-1}^{fast} \mathbf{k}_t$$

biểu diễn phần sai khác giữa giá trị cần ghi và giá trị hiện tại mà bộ nhớ dự đoán từ khóa. Vì vậy, bộ nhớ nhanh không ghi đè toàn bộ trạng thái cũ, mà chỉ cập nhật theo phần sai khác của đặc trưng mới. Trong công thức này, $(1 - \alpha)$ điều khiển mức giữ lại trạng thái cũ, α có thể được hiểu như hệ số quên, còn η_t điều khiển cường độ cập nhật theo embedding hiện tại.

Sau khi qua Fast Memory, biểu diễn được đưa vào Slow Memory. Slow Memory được tổ chức dưới dạng coreset hợp nhất:

$$\mathcal{C}_{\text{patch}}^{(t)} = \text{Select}_M \left(\mathcal{C}_{\text{patch}}^{(t-1)} \cup \mathcal{H}_t \right), \quad \left| \mathcal{C}_{\text{patch}}^{(t)} \right| \leq M,$$

trong đó $\mathcal{H}_t$ là tập đặc trưng sau khi được xử lý bởi Fast Memory, còn $\text{Select}_M(\cdot)$ là bước chọn lại tối đa M đặc trưng đại diện. Công thức này thể hiện vai trò của bộ nhớ chậm: không cập nhật theo mọi biến động nhỏ, mà giữ một tập đặc trưng bình thường có tính đại diện qua nhiều tác vụ.

Như vậy, mô hình có hai mức cập nhật rõ ràng. Fast Memory phản ứng nhanh với đặc trưng mới, còn Slow Memory giữ các đại diện ổn định hơn để phục vụ tính điểm bất

thường. Cách tổ chức này phù hợp với bài toán phát hiện bất thường liên tục, vì hệ thống cần tiếp nhận dữ liệu bình thường mới nhưng vẫn hạn chế làm mất các đại diện quan trọng của dữ liệu cũ.

3.5 Chỉ số đánh giá

Trong bài toán phát hiện bất thường liên tục, mô hình cần được đánh giá ở ba khía cạnh: khả năng phát hiện ảnh bất thường, khả năng định vị vùng bất thường và khả năng duy trì hiệu năng trên các tác vụ đã học. Do đó, nhóm chỉ số được sử dụng gồm image-level AUROC, pixel-level AUROC/AUPR và Forgetting Measure (FM). Các chỉ số cấp ảnh phản ánh khả năng phân biệt ảnh bình thường và ảnh bất thường; các chỉ số cấp điểm ảnh phản ánh khả năng định vị vùng lỗi; còn FM cho biết mức suy giảm hiệu năng sau khi mô hình học thêm các tác vụ mới.

3.5.1 Đánh giá cấp ảnh và cấp điểm ảnh

Image-level AUROC.

Ở mức độ ảnh, mỗi ảnh kiểm thử được gán một điểm bất thường s_{img} . Điểm này càng lớn thì ảnh càng có khả năng chứa lỗi. Để đánh giá khả năng phân biệt giữa ảnh bình thường và ảnh bất thường mà không phụ thuộc vào một ngưỡng cố định, ta sử dụng chỉ số AUROC ở cấp ảnh.

Với mỗi ngưỡng phân loại, các ảnh được chia thành hai nhóm bình thường và bất thường. Khi đó, hai đại lượng cơ bản của đường ROC là:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}.$$

Trong đó, TP là số ảnh bất thường được phát hiện đúng, FN là số ảnh bất thường bị bỏ sót, FP là số ảnh bình thường bị dự đoán nhầm thành bất thường, và TN là số ảnh bình thường được nhận diện đúng. Đường ROC được tạo bằng cách thay đổi ngưỡng phân loại và ghi nhận các cặp giá trị (FPR, TPR). Chỉ số AUROC là diện tích dưới đường cong này:

$$\text{AUROC} = \int_0^1 \text{TPR}(\text{FPR}) d\text{FPR}.$$

Giá trị AUROC càng cao cho thấy mô hình càng có khả năng xếp hạng ảnh bất thường

cao hơn ảnh bình thường. Trong thực nghiệm, image-level AUROC được dùng để đánh giá năng lực phát hiện ảnh lỗi của mô hình.

Pixel-level AUROC.

Ở mức độ điểm ảnh, mô hình tạo ra một bản đồ điểm bất thường cho ảnh đầu vào. Mỗi điểm ảnh hoặc mỗi vị trí trên bản đồ bất thường được xem như một mẫu đánh giá riêng. Điểm bất thường tại vị trí đó được so sánh với mặt nạ lỗi ground-truth để xác định điểm ảnh bình thường hay bất thường.

Tương tự cấp ảnh, pixel-level AUROC được tính bằng cách thay đổi ngưỡng trên bản đồ điểm bất thường và xây dựng đường ROC ở cấp điểm ảnh. Với mỗi ngưỡng, ta tính:

$$\text{TPR}_{\text{pix}} = \frac{\text{TP}_{\text{pix}}}{\text{TP}_{\text{pix}} + \text{FN}_{\text{pix}}}, \quad \text{FPR}_{\text{pix}} = \frac{\text{FP}_{\text{pix}}}{\text{FP}_{\text{pix}} + \text{TN}_{\text{pix}}}.$$

Trong đó, các đại lượng TP_{pix} , FP_{pix} , FN_{pix} và TN_{pix} được tính trên toàn bộ điểm ảnh của tập kiểm thử. Pixel-level AUROC được xác định bởi:

$$\text{AUROC}_{\text{pix}} = \int_0^1 \text{TPR}_{\text{pix}}(\text{FPR}_{\text{pix}}) d\text{FPR}_{\text{pix}}.$$

Chỉ số này cho biết mô hình có gán điểm bất thường cao hơn cho các điểm ảnh lỗi so với các điểm ảnh bình thường hay không. Tuy nhiên, trong ảnh công nghiệp, vùng lỗi thường chiếm tỉ lệ nhỏ, nên pixel-level AUROC có thể chưa phản ánh đầy đủ chất lượng định vị lỗi trong trường hợp mất cân bằng mạnh giữa hai lớp điểm ảnh.

Pixel-level AUPR/AP.

Để đánh giá rõ hơn khả năng định vị vùng lỗi trong điều kiện số điểm ảnh bất thường nhỏ hơn nhiều so với số điểm ảnh bình thường, ta sử dụng thêm pixel-level AUPR. Với một ngưỡng nhị phân hóa bản đồ bất thường, precision và recall ở cấp điểm ảnh được tính bởi:

$$\text{Precision} = \frac{\text{TP}_{\text{pix}}}{\text{TP}_{\text{pix}} + \text{FP}_{\text{pix}}}, \quad \text{Recall} = \frac{\text{TP}_{\text{pix}}}{\text{TP}_{\text{pix}} + \text{FN}_{\text{pix}}}.$$

Đường precision–recall được xây dựng bằng cách thay đổi ngưỡng trên bản đồ điểm bất thường. Khi đó, AUPR được tính là diện tích dưới đường cong precision–recall:

$$\text{AUPR} = \int_0^1 \text{Precision}(\text{Recall}) d\text{Recall}.$$

Trong thực nghiệm, AUPR thường được xấp xỉ bằng Average Precision (AP), bằng cách sắp xếp toàn bộ điểm ảnh theo điểm bất thường giảm dần và tổng hợp precision tại các mức recall khác nhau. Giá trị pixel-level AUPR càng cao cho thấy mô hình càng tập trung tốt điểm bất thường vào đúng vùng lỗi, thay vì tạo nhiều cảnh báo sai trên nền bình thường.

3.5.2 Đánh giá mức độ quên

Forgetting Measure.

Để đo mức độ suy giảm hiệu năng trên các tác vụ cũ, ta sử dụng Forgetting Measure (FM) [12].

Gọi $a_{t,i}$ là hiệu năng của mô hình trên tác vụ i sau khi đã học xong đến tác vụ t , với $i \leq t$. Đại lượng $a_{t,i}$ có thể là image-level AUROC, pixel-level AUROC hoặc pixel-level AUPR. Với một tác vụ cũ $i < t$, mức quên sau khi học đến tác vụ t được tính bằng chênh lệch giữa hiệu năng tốt nhất từng đạt được trên tác vụ đó và hiệu năng hiện tại:

$$f_{t,i} = \max_{l \in \{i, \dots, t-1\}} a_{l,i} - a_{t,i}, \quad i < t.$$

Mức quên trung bình sau khi học đến tác vụ t là:

$$\text{FM}_t = \frac{1}{t-1} \sum_{i=1}^{t-1} f_{t,i}.$$

Sau khi hoàn tất chuỗi gồm T tác vụ, mức quên cuối cùng được lấy là:

$$\text{FM} = \text{FM}_T.$$

Giá trị FM càng nhỏ cho thấy mô hình càng giữ được hiệu năng trên các tác vụ cũ. Trong đồ án này, FM được tính riêng cho image-level AUROC và pixel-level AUPR để xác định mô hình quên nhiều hơn ở khâu phát hiện ảnh bất thường hay ở khâu định vị vùng lỗi.

Hiệu năng trung bình sau chuỗi học liên tục.

Bên cạnh FM, cần báo cáo hiệu năng trung bình của mô hình sau khi học xong toàn bộ chuỗi tác vụ. Với một chỉ số đánh giá bất kỳ m , chẳng hạn image-level AUROC hoặc pixel-level AUPR, hiệu năng trung bình cuối cùng được tính bởi:

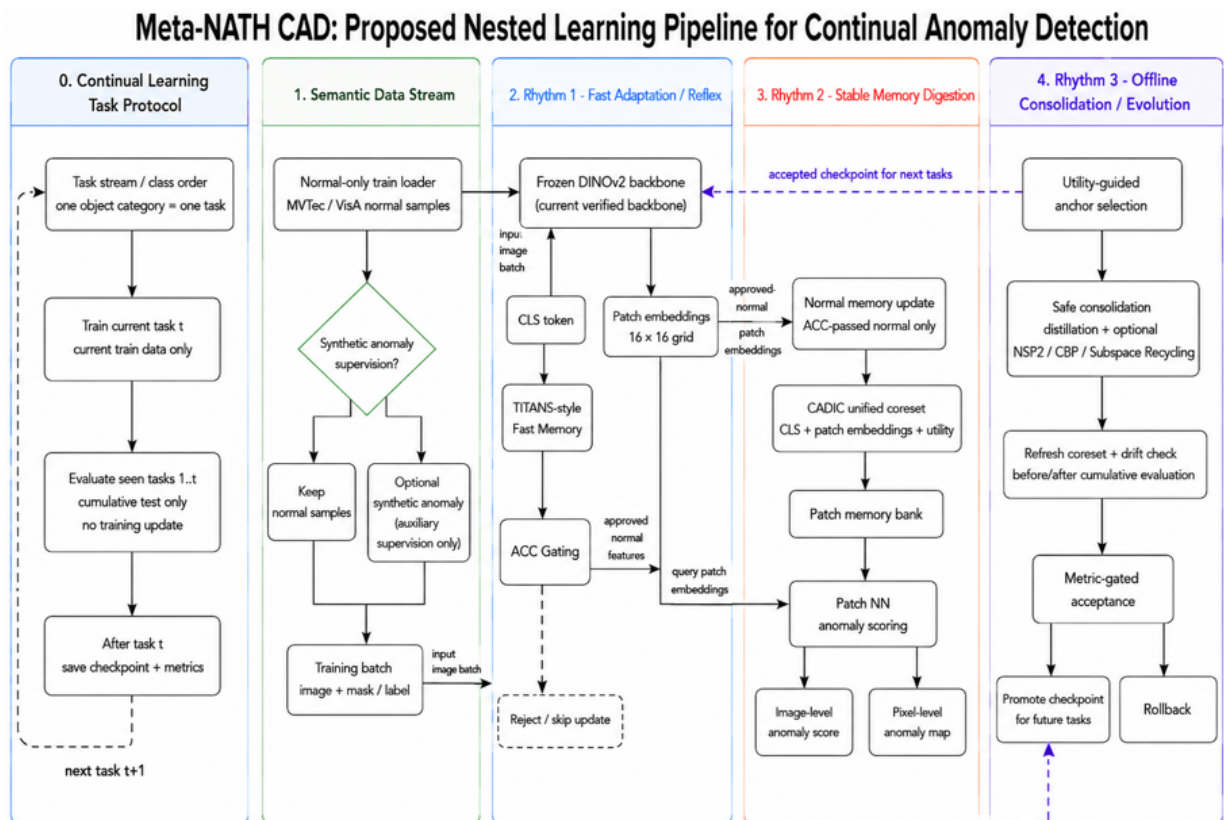
$$\bar{m} = \frac{1}{T} \sum_{i=1}^T a_{T,i}.$$

Trong đó, $a_{T,i}$ là hiệu năng trên tác vụ i sau khi mô hình đã học xong toàn bộ T tác vụ. Chỉ số $\bar{m}$ cho biết hiệu năng còn lại của mô hình ở cuối quá trình học, còn FM cho biết mức suy giảm so với thời điểm mô hình đạt kết quả tốt nhất trên từng tác vụ. Hai đại lượng này cần được xem xét cùng nhau: $\bar{m}$ cao cho thấy mô hình còn giữ được hiệu năng tổng thể tốt, còn FM thấp cho thấy mô hình ít bị suy giảm trên các tác vụ cũ.

Chương 4

Giải pháp đề xuất

4.1 Tổng quan phương pháp



Hình 6: Pipeline tổng thể của giải pháp CAD được đề xuất, gồm ba giai đoạn chính: Thích nghi nhanh, tiêu hóa tri thức và củng cố offline (Tiến hoá).

Từ các phân tích ở những chương trước, có thể thấy bài toán CAD trong ảnh công nghiệp đòi hỏi hệ thống phải đồng thời thỏa mãn hai yêu cầu cơ bản:

- **Một là**, mô hình cần duy trì khả năng phát hiện và định vị bất thường trên các nhóm sản phẩm đã học trước đó nhằm hạn chế hiện tượng catastrophic forgetting.
- **Hai là**, mô hình vẫn phải có đủ tính plasticity để tiếp nhận các nhóm sản phẩm mới hoặc các distribution shift (data drift) phát sinh trong quá trình vận hành.

Để giải quyết hai yêu cầu trên, nhóm đề xuất một pipeline CAD theo định hướng Nested

Learning [18]. Thay vì xem mô hình như một khối duy nhất được huấn luyện lại sau mỗi tác vụ, hệ thống được thiết kế thành nhiều layers có tốc độ cập nhật và vai trò khác nhau:

- **Tầng thứ nhất:** Đảm nhiệm việc trích xuất đặc trưng và thích nghi nhanh với dữ liệu đầu vào.
- **Tầng thứ hai:** Duy trì một bộ nhớ ổn định dưới dạng coreset đặc trưng bình thường.
- **Tầng thứ ba:** Thực hiện quá trình củng cố offline có kiểm soát, trong đó một số mẫu neo có giá trị đại diện cao được sử dụng lại để điều chỉnh mô hình mà không làm suy giảm đáng kể hiệu năng tích lũy.

Về mặt trực giác, cách tổ chức này tương tự một hệ thống có nhiều mức bộ nhớ. Lấy cảm hứng từ những nghiên cứu về não bộ và cách não hoạt động, thực chất, bộ não không chỉ có một tần suất cập nhật tri thức mà có nhiều nấc. Các cập nhật nhanh giúp mô hình phản ứng với dữ liệu mới, bộ nhớ ổn định lưu giữ các biểu diễn đại diện của phân phối bình thường, bước củng cố chậm đóng vai trò điều chỉnh lại biểu diễn ở mức backbone khi có đủ bằng chứng. Cấu trúc nhiều tầng này phù hợp với tinh thần gốc của Nested Learning, trong đó quá trình học được phân rã thành các vòng lặp con có vai trò khác nhau thay vì chỉ tối ưu một Loss Function duy nhất trên toàn bộ mô hình [18].

Pipeline đề xuất gồm ba giai đoạn chính:

- **Giai đoạn thứ nhất - thích nghi nhanh:** Sử dụng backbone thị giác đã huấn luyện trước để trích xuất đặc trưng ảnh, sau đó đưa biểu diễn toàn cục qua cơ chế bộ nhớ nhanh theo hướng TITANS/TTA [20].
- **Giai đoạn thứ hai - tiêu hóa tri thức:** Sử dụng ACC gating để quyết định có cập nhật mẫu hiện tại vào bộ nhớ ổn định hay không, sau đó lưu các embedding bình thường vào CADIC coreset [15].
- **Giai đoạn thứ ba - củng cố:** Được thực hiện offline hoặc trên cloud bằng cách sử dụng các anchor có utility score cao trong coreset để điều chỉnh nhẹ backbone, làm mới embedding trong coreset và chỉ chấp nhận checkpoint mới bằng metric-gated acceptance nếu các chỉ số tích lũy không suy giảm quá ngưỡng.

Trong phạm vi kết quả chính của đề án, backbone được sử dụng là DINOv2 do tính ổn định và khả năng trích xuất đặc trưng tổng quát tốt trên nhiều tác vụ thị giác. DINOv2 là một mô hình thị giác tự giám sát được thiết kế nhằm tạo ra các đặc trưng tổng quát, có thể sử dụng lại trên nhiều bài toán ở cả cấp độ ảnh và cấp độ điểm ảnh [21]. Điều này

phù hợp với bài toán phát hiện bất thường công nghiệp, nơi dữ liệu lỗi thường hiếm và mô hình cần tận dụng biểu diễn thị giác đã học từ dữ liệu lớn. Trong thiết kế tổng quát, pipeline có thể mở rộng sang các backbone mới hơn như DINOv3, tuy nhiên, các kết quả thực nghiệm chính của đề án được thực hiện trên DINOv2 để đảm bảo khả năng tái lập và ổn định và nhóm cũng sẽ mở rộng adapter sang cả v3 sau khi đã ổn định v2.

4.2 Trích xuất biểu diễn ảnh bằng Vision Backbone

Đầu vào của hệ thống là một ảnh công nghiệp $x \in \mathbb{R}^{H \times W \times 3}$. Ảnh được đưa qua backbone thị giác đã huấn luyện trước để thu được hai loại biểu diễn: Embedding toàn cục của ảnh và các embedding cục bộ theo patch. Với backbone kiểu Vision Transformer, ảnh được chia thành các patch, sau đó, mỗi patch được biểu diễn bằng một vector đặc trưng trong không gian d -chiều. Ký hiệu đầu ra của backbone là:

$$z_{\text{cls}} = f_{\theta}^{\text{cls}}(x) \in \mathbb{R}^d,$$

$$Z_{\text{patch}} = f_{\theta}^{\text{patch}}(x) \in \mathbb{R}^{N \times d},$$

trong đó z_{cls} là embedding toàn cục, Z_{patch} là tập N embedding patch, và d là số chiều đặc trưng. Trong thực nghiệm của đề án, với ảnh đầu vào kích thước 224×224 và backbone DINOv2-base, số patch là $N = 256$, tương ứng với lưới patch 16×16 .

Embedding toàn cục cung cấp thông tin ở cấp độ ảnh, phù hợp cho nhiệm vụ phát hiện ảnh bất thường. Ngược lại, patch embedding giữ lại cấu trúc không gian cục bộ, giúp xây dựng bản đồ bất thường ở mức điểm ảnh thông qua so khớp lân cận gần nhất. Cách tổ chức này kế thừa tinh thần của các phương pháp dựa trên memory bank như PatchCore, nhưng được mở rộng cho thiết lập Continual Learning bằng cách cập nhật coreset theo dòng tác vụ thay vì xây dựng memory bank một lần trong thiết lập cố định [3].

Trong giai đoạn Phase 1-2, backbone được giữ cố định nhằm tránh làm dịch chuyển mạnh không gian đặc trưng sau mỗi tác vụ. Đây là một lựa chọn có chủ đích trong bối cảnh Continual Learning: Nếu cập nhật trực tiếp toàn bộ backbone trên dữ liệu mới, mô hình có nguy cơ phá vỡ các biểu diễn đã có ích cho những tác vụ cũ. Thay vào đó, hệ thống ưu tiên cập nhật các cơ chế bộ nhớ và coreset, trong khi việc tinh chỉnh backbone chỉ được thực hiện ở Phase 3 với cơ chế kiểm soát chặt chẽ khi đã tích lũy được lượng tri thức tiềm năng từ Phase 1-2.

4.3 Cơ chế Fast Memory TITANS và Test-Time Adaptation

Sau khi thu được embedding toàn cục z_{cls} , hệ thống đưa vector này qua một module Fast Memory. Module này được lấy cảm hứng từ hướng tiếp cận TITANS, trong đó mô hình duy trì một bộ nhớ có thể cập nhật ở thời điểm suy luận để hấp thụ thông tin từ ngữ cảnh mới gọi là TTA [20]. Trong đề án, Fast Memory được hiện thực bằng một ma trận $M \in \mathbb{R}^{d \times d}$, đóng vai trò lưu giữ biến đổi tuyến tính ngắn hạn trên không gian đặc trưng.

Với embedding đầu vào z , bộ nhớ tạo ra một biểu diễn truy xuất:

$$\hat{z} = zM.$$

Sau đó, bộ nhớ được cập nhật bằng một dạng delta rule dựa trên sai khác giữa biểu diễn hiện tại và biểu diễn dự đoán. Khái quát hơn, cập nhật có thể viết dưới dạng:

$$M \leftarrow M + \eta \Delta M,$$

Trong đó, η là tốc độ cập nhật và ΔM được xây dựng từ tín hiệu sai khác giữa embedding hiện tại và trạng thái bộ nhớ. Đồng thời, hệ thống ghi nhận một đại lượng surprise scalar, phản ánh mức độ khác biệt giữa dữ liệu hiện tại và trạng thái bộ nhớ đã có. Khi surprise cao, dữ liệu hiện tại có thể biểu hiện distribution shift hoặc dạng mẫu chưa quen thuộc; khi surprise thấp, mẫu hiện tại gần với trạng thái đã được bộ nhớ hấp thụ.

Vai trò của Fast Memory không phải là thay thế backbone mà là tạo ra một tầng thích nghi mềm ở phía sau backbone. Trong môi trường công nghiệp, dữ liệu có thể thay đổi do chiếu sáng, góc chụp, độ mòn thiết bị hoặc dao động nhỏ trong quy trình sản xuất,... Những thay đổi này chưa nhất thiết là bất thường về sản phẩm, nhưng có thể làm phân phối đặc trưng dịch chuyển nhẹ. Fast Memory cho phép hệ thống phản ứng với các thay đổi như vậy mà không cần huấn luyện lại toàn bộ mô hình.

Tuy nhiên, việc cho phép mô hình thích nghi liên tục cũng đặt ra rủi ro: Nếu mọi mẫu đều được dùng để cập nhật bộ nhớ ổn định, hệ thống có thể học nhầm cả mẫu lỗi hoặc mẫu nhiễu. Vì vậy, đầu ra của bộ nhớ nhanh không được đưa trực tiếp vào coresets, mà phải đi qua một cơ chế lọc được trình bày ở phần kế tiếp.

4.4 ACC Gating và kiểm soát cập nhật bộ nhớ

ACC Gating được sử dụng như một cơ chế kiểm soát trước khi cập nhật tri thức mới vào bộ nhớ ổn định. Ý tưởng chính là so sánh biểu diễn sau khi đi qua bộ nhớ nhanh với biểu diễn ban đầu từ backbone. Nếu hai biểu diễn còn đủ nhất quán, mẫu hiện tại được xem là hợp lệ để cập nhật vào coreset. Ngược lại, nếu sự thay đổi quá lớn, hệ thống có thể từ chối cập nhật để tránh đưa dữ liệu không đáng tin cậy vào bộ nhớ.

Ký hiệu z_{cls} là embedding ban đầu và z_{upd} là embedding sau bộ nhớ nhanh. Một điểm nhất quán có thể được tính dựa trên cosine similarity:

$$ACC(z_{cls}, z_{upd}) = \frac{z_{cls}^T z_{upd}}{\|z_{cls}\|_2 \|z_{upd}\|_2}.$$

Quyết định cập nhật được xác định bởi ngưỡng τ_{acc} :

$$approve(x) = \begin{cases} 1, & \text{nếu } ACC(z_{cls}, z_{upd}) \geq \tau_{acc}, \\ 0, & \text{ngược lại.} \end{cases}$$

Về mặt thiết kế, ACC Gating đóng vai trò làm giảm nguy cơ cập nhật sai vào slow memory. Trong bài toán CAD, tập huấn luyện của mỗi tác vụ chủ yếu gồm ảnh bình thường, nhưng trong quá trình sinh bất thường giả hoặc khi dữ liệu bị nhiễu, vẫn có khả năng xuất hiện các mẫu không nên được xem là đại diện bình thường. Do đó, ngoài việc chỉ cập nhật từ các mẫu được xác định là normal trong loader, hệ thống còn sử dụng gating để hạn chế việc đưa các biểu diễn không ổn định vào coreset.

Trong thực nghiệm hiện tại, ngưỡng τ_{acc} được đặt ở mức bảo thủ để tránh việc từ chối toàn bộ một số lớp khó. Kết quả quan sát cho thấy với ngưỡng quá cao, một số tác vụ có thể không được cập nhật đủ vào coreset, làm suy giảm mạnh khả năng phát hiện bất thường. Vì vậy, đề án sử dụng $\tau_{acc} = 0.6$ cho đường chạy chính. Ở cấu hình này, ACC gating hoạt động như một bộ lọc ổn định nhưng chưa phải là một detector hoàn chỉnh cho mọi dạng distribution shift. Việc hiệu chỉnh ACC theo thống kê từng lớp hoặc từng batch được xem là một trong những hướng cải thiện trong tương lai.

4.5 CADIC coreset và bộ nhớ ổn định

Sau khi mẫu được ACC gating chấp nhận, hệ thống cập nhật mẫu vào CADIC coreset [15]. coreset đóng vai trò là bộ nhớ ổn định của pipeline, lưu giữ các biểu diễn đại diện của phân phối bình thường qua nhiều tasks liên tiếp. Khác với việc lưu toàn bộ ảnh hoặc toàn bộ patch embedding của mọi ảnh huấn luyện, coreset giới hạn kích thước bộ nhớ bằng một ngưỡng K . Điều này giúp kiểm soát chi phí lưu trữ và chi phí tìm kiếm lân cận gần nhất khi số tác vụ tăng dần.

Mỗi phần tử trong coreset gồm các thành phần chính: Embedding toàn cục, patch embedding, mã tác vụ, utility score và tùy chọn ảnh anchor trong Phase 3. Có thể biểu diễn một entry của coreset dưới dạng:

$$e_i = (z_i^{\text{cls}}, Z_i^{\text{patch}}, t_i, u_i, x_i),$$

Trong đó t_i là chỉ số tác vụ, u_i là utility score và x_i là ảnh anchor nếu hệ thống được cấu hình lưu ảnh cho Phase 3. Trong Phase 1-2, coreset chủ yếu phục vụ scoring; trong Phase 3, một số entry có utility score cao được chọn lại làm anchor replay để củng cố mô hình.

Cơ chế thay thế trong coreset được thiết kế nhằm giữ các mẫu có ích cho việc bao phủ không gian đặc trưng bình thường. Khi coreset chưa đầy, các mẫu được thêm trực tiếp. Khi coreset đạt kích thước tối đa, hệ thống cần quyết định mẫu nào nên được giữ và mẫu nào có thể bị thay thế. Mục tiêu là coreset nên giữ được các đại diện sao cho mọi mẫu bình thường từng quan sát đều có thể được xấp xỉ bởi một phần tử gần trong bộ nhớ. Cách làm này tương đồng với coreset subsampling trong PatchCore, nơi mục tiêu là giảm kích thước memory bank nhưng vẫn bảo toàn khả năng bao phủ không gian patch embedding [3].

Cơ chế coreset giúp giảm catastrophic forgetting theo hai cách:

- **Thứ nhất**, nó giữ lại đại diện của các tác vụ cũ, nhờ đó scoring ở giai đoạn kiểm thử vẫn có thể so sánh ảnh mới với phân phối bình thường đã học trước đó.
- **Thứ hai**, coreset không tăng trưởng tuyến tính theo toàn bộ dữ liệu lịch sử, giúp hệ thống khả thi hơn trong thiết lập công nghiệp có nhiều vòng cập nhật.

Tuy nhiên, coreset cũng có giới hạn tự nhiên: Nếu chỉ lưu embedding mà không lưu đủ chi tiết ảnh, khả năng định vị lỗi nhỏ ở mức pixel có thể kém hơn các phương pháp phát lại dữ liệu có độ trung thực cao như ReplayCAD.

4.6 Utility-guided anchor replay

Trong thiết kế, hệ thống không loại bỏ hoàn toàn replay mà sử dụng một dạng anchor replay nhẹ dựa trên utility score. Cụ thể, thay vì dùng generative replay để sinh lại dữ liệu lịch sử như ReplayCAD, hệ thống chọn một số mẫu anchor có utility score cao từ coreset để sử dụng trong Phase 3. Cách tiếp cận này được gọi là utility-guided anchor replay.

Về mặt trực giác, nếu một hệ thống chỉ giữ các vector embedding mà không giữ lại bất kỳ ví dụ hình ảnh nào, nó có thể vẫn nhớ được cấu trúc tổng thể của phân phối bình thường, nhưng sẽ khó duy trì chi tiết cục bộ cần thiết cho định vị lỗi nhỏ. Điều này tương tự việc con người có thể nhớ khái quát về một đối tượng đã nhìn thấy, nhưng nếu không xem lại hình ảnh cụ thể, các chi tiết nhỏ có thể bị mờ dần. Do đó, việc lưu lại một số anchor đại diện là hợp lý trong bối cảnh cần sự cân bằng giữa chi phí lưu trữ và khả năng bảo toàn chi tiết pixel.

Khác với ReplayCAD, anchor replay trong đồ án không sử dụng mô hình khuếch tán để sinh lại dữ liệu cũ. ReplayCAD đạt hiệu quả cao trong segmentation bằng cách nén dữ liệu lịch sử thành các điều kiện semantic và spatial cho mô hình khuếch tán, sau đó phát lại dữ liệu có chi tiết pixel nhằm giảm quên và cải thiện định vị bất thường [16]. Đây là một hướng rất mạnh cho pixel-level CAD, nhưng cũng làm pipeline phụ thuộc vào mô hình sinh và quá trình phát lại dữ liệu phức tạp. Ngược lại, utility-guided anchor replay chỉ lưu một số mẫu đại diện trong coreset, dùng chúng để củng cố offline, distillation và kiểm chứng checkpoint.

Trong Phase 3, các anchor được chọn theo utility score và được cân bằng theo tác vụ. Việc cân bằng theo tác vụ là cần thiết vì nếu chỉ chọn các mẫu có utility score cao toàn cục, coreset có thể ưu tiên một số lớp dễ hoặc một số lớp có nhiều mẫu, làm giảm khả năng bảo vệ các lớp ít đại diện. Ký hiệu $\mathcal{A}$ là tập anchor được chọn, ta có:

$$\mathcal{A} = \text{TopK}_{\text{balanced}}(\mathcal{C}, u_i, t_i),$$

Trong đó $\mathcal{C}$ là coreset, u_i là utility score và t_i là chỉ số tác vụ của entry thứ i . Tập anchor này sau đó được sử dụng để so sánh biểu diễn trước và sau khi tinh chỉnh backbone, đồng thời làm nguồn dữ liệu cho patch-token distillation.

Cách tiếp cận này tạo ra một điểm trung gian. Nếu không dùng replay, hệ thống có thể thiếu chi tiết pixel của các tác vụ cũ. Nếu dùng generative replay đầy đủ, pipeline có thể

phức tạp nặng nề và khó kiểm soát hơn trong triển khai. Utility-guided anchor replay giữ lại một lượng nhỏ ví dụ đại diện, đủ để hỗ trợ củng cố offline, nhưng không biến toàn bộ pipeline thành một hệ thống generative replay.

4.7 Tính điểm bất thường bằng patch nearest-neighbor

Sau khi coreset được xây dựng, quá trình suy luận được thực hiện bằng cách so khớp patch embedding của ảnh kiểm thử với patch embedding bình thường trong coreset. Với ảnh kiểm thử x , backbone trích xuất tập patch embedding:

$$Z_{\text{test}} = \{z_1, z_2, \dots, z_N\}.$$

Gọi $\mathcal{M}$ là patch bank gồm toàn bộ patch embedding bình thường được lưu trong coreset. Điểm bất thường của patch z_i được tính bằng khoảng cách đến các lân cận gần nhất trong $\mathcal{M}$. Ở dạng đơn giản nhất:

$$s_i = \min_{m \in \mathcal{M}} d(z_i, m),$$

Trong đó $d(\cdot, \cdot)$ là khoảng cách trong không gian embedding. Trong hiện thực của đề án, số lân cận gần nhất được cấu hình bởi tham số b , với giá trị chính là $b = 9$. Việc dùng nhiều lân cận thay vì chỉ một lân cận giúp điểm số ổn định hơn trước các nhiễu nhỏ trong memory bank.

Tập điểm patch $\{s_i\}_{i=1}^N$ được reshape về lưới patch 16×16 , sau đó nội suy về kích thước ảnh gốc để tạo bản đồ bất thường. Điểm bất thường cấp ảnh s_{img} được suy ra từ các điểm patch có giá trị cao. Cách làm này phù hợp với các phương pháp memory bank như PatchCore và được mở rộng trong CADIC cho thiết lập phát hiện bất thường liên tục [3, 15].

Cách tính điểm này có ưu điểm là không cần huấn luyện bộ phân loại bất thường có giám sát. Hệ thống chỉ cần ảnh bình thường để xây dựng coreset, sau đó phát hiện bất thường bằng độ lệch hình học trong không gian embedding. Tuy nhiên, chất lượng định vị pixel phụ thuộc mạnh vào độ bao phủ của coreset và độ ổn định của score distribution giữa các lớp. Khi số tác vụ tăng, một patch bank toàn cục có thể gặp xung đột phân phối giữa nhiều lớp, làm giảm cumulative pixel AUPR. Đây là một hạn chế được quan sát trong thực nghiệm và sẽ được thảo luận ở Chương 6.

4.8 Phase 3: Củng cố offline có kiểm soát

Phase 3 được thiết kế nhằm thực hiện bước củng cố chậm sau khi hệ thống đã học qua một số tác vụ. Khác với Phase 1-2, nơi backbone được giữ cố định và hệ thống chủ yếu cập nhật coreset, Phase 3 cho phép tinh chỉnh một phần nhỏ của backbone. Tuy nhiên, việc cập nhật này được thực hiện theo hướng bảo thủ và có kiểm soát để tránh phá vỡ không gian embedding đã dùng cho các tác vụ cũ.

Đầu vào của Phase 3 là checkpoint sau quá trình warmup và tập anchor được chọn từ coreset. Hệ thống chỉ mở khóa một số block cuối của backbone, trong khi phần lớn backbone vẫn được giữ cố định. Với ảnh anchor x_i , hệ thống lưu lại biểu diễn trước khi cập nhật làm teacher:

$$z_i^{\text{old}} = f_{\theta_{\text{old}}}(x_i),$$

Và sau đó tối ưu backbone mới sao cho biểu diễn sau cập nhật không lệch quá xa khỏi teacher:

$$\mathcal{L}_{\text{cls}} = \frac{1}{|\mathcal{A}|} \sum_{x_i \in \mathcal{A}} \left\| f_{\theta}^{\text{cls}}(x_i) - z_i^{\text{old, cls}} \right\|_2^2.$$

Bên cạnh distillation ở cấp độ CLS token, hệ thống còn sử dụng patch-token distillation:

$$\mathcal{L}_{\text{patch}} = \frac{1}{|\mathcal{A}|N} \sum_{x_i \in \mathcal{A}} \sum_{j=1}^N \left\| f_{\theta}^{\text{patch}}(x_i)_j - z_{i,j}^{\text{old, patch}} \right\|_2^2.$$

Hàm mất mát tổng quát của Phase 3 có dạng:

$$\mathcal{L}_{\text{phase3}} = \lambda_{\text{cls}} \mathcal{L}_{\text{cls}} + \lambda_{\text{patch}} \mathcal{L}_{\text{patch}} + \lambda_{\text{lejepa}} \mathcal{L}_{\text{lejepa}},$$

trong đó $\mathcal{L}_{\text{lejepa}}$ là một surrogate loss lấy cảm hứng từ họ Joint-Embedding Predictive Architecture, trong đó mô hình học bằng cách dự đoán biểu diễn tiềm ẩn thay vì tái tạo trực tiếp pixel [22]. Trong cấu hình bảo thủ được dùng cho kết quả chính, trọng số LEJEPa được đặt bằng 0, trong khi patch distillation được đặt cao hơn để ưu tiên bảo toàn không gian patch phục vụ pixel-level localization.

Sau cập nhật, hệ thống đo độ lệch biểu diễn giữa trước và sau cập nhật. Nếu drift vượt quá ngưỡng cho phép, checkpoint bị rollback. Nếu không rollback, toàn bộ embedding trong coreset được refresh bằng backbone mới. Bước refresh này rất quan trọng: Nếu backbone đã thay đổi nhưng coreset vẫn lưu embedding cũ, ảnh kiểm thử sẽ được biểu diễn trong không gian mới trong khi memory bank vẫn nằm trong không gian cũ, làm sai lệch kết quả nearest-neighbor.

Cuối cùng, checkpoint Phase 3 không được tự động chấp nhận chỉ vì loss giảm hoặc drift nhỏ. Hệ thống thực hiện before/after evaluation và dùng metric-gated acceptance. Cụ thể, checkpoint mới chỉ được chấp nhận nếu final cumulative pixel AUPR không giảm quá ngưỡng cho phép và final cumulative image AUROC không suy giảm quá ngưỡng bảo vệ. Đây là một điểm quan trọng trong thiết kế vì nó biến Phase 3 từ một bước cập nhật mù thành một cơ chế củng cố có kiểm chứng.

4.9 Vai trò của NSP2 và CBP trong thiết kế mở rộng

Bên cạnh pipeline chính, nhóm cũng hiện thực các module NSP2 và CBP như các cơ chế mở rộng cho Phase 3. NSP2, viết tắt của Null-Space Projection, được thiết kế nhằm giảm interference khi cập nhật backbone [23]. Ý tưởng chính là ước lượng một không gian con liên quan đến tri thức cũ, sau đó chiếu gradient cập nhật vào không gian trực giao hoặc không gian ít ảnh hưởng đến các hướng biểu diễn cũ. Nếu g là gradient cần cập nhật và $P_{\mathcal{N}}$ là phép chiếu vào null space, gradient sau khi bảo vệ có thể viết là:

$$g' = P_{\mathcal{N}} g.$$

NSP2 không phải là một cơ chế replay và cũng không tự khôi phục chi tiết ảnh cũ. Vai trò của nó là bảo vệ hướng cập nhật khi hệ thống học từ anchor replay hoặc khi backbone được tinh chỉnh. Nói cách khác, anchor replay cung cấp các ví dụ đại diện để mô hình xem lại, còn NSP2 giúp quá trình cập nhật không phá vỡ các hướng biểu diễn quan trọng đã hình thành trước đó.

CBP, hay Continual Backpropagation, được đưa vào như một cơ chế theo dõi và tái khởi tạo các đơn vị có utility score thấp hoặc có dấu hiệu chết [24]. Trong các mạng học sâu cập nhật liên tục, một số neuron có thể trở nên kém hoạt động hoặc không còn đóng góp đáng kể cho đầu ra. CBP hướng đến việc duy trì khả năng thích nghi lâu dài bằng cách phát hiện các đơn vị như vậy và tái khởi tạo khi cần thiết. Trong kết quả chính của đồ

án, CBP được xem là module thực nghiệm; hệ thống có kiểm thử đường chạy nhưng chưa dùng nó làm cơ sở để khẳng định cải thiện benchmark.

Như vậy, trong thiết kế tổng thể, ba cơ chế bảo vệ tri thức cũ có vai trò bổ trợ nhau: CADIC coreset lưu tri thức cũ ở dạng feature memory, utility-guided anchor replay giữ lại một số ví dụ đại diện để củng cố offline và NSP2 bảo vệ hướng cập nhật khi tinh chỉnh backbone. Bên ngoài các cơ chế này, metric-gated acceptance đóng vai trò lớp an toàn triển khai, bảo đảm checkpoint mới không được đưa vào sử dụng nếu làm suy giảm hiệu năng tích lũy quá ngưỡng.

Chương 5

Thực nghiệm

5.1 Mục tiêu thực nghiệm

Mục tiêu của phần thực nghiệm là đánh giá pipeline đề xuất trong thiết lập CAD trên ảnh công nghiệp. Cụ thể, thực nghiệm tập trung trả lời bốn câu hỏi:

- **Thứ nhất**, hệ thống có thể học tuần tự qua nhiều nhóm dữ liệu trong MVTec AD mà không bị lỗi pipeline hoặc tăng trưởng bộ nhớ ngoài kiểm soát hay không. Đây là yêu cầu cơ bản đối với một hệ thống Continual Learning vì mô hình phải xử lý dữ liệu theo chuỗi tác vụ thay vì huấn luyện một lần trên toàn bộ tập dữ liệu.
- **Thứ hai**, hệ thống có duy trì được hiệu năng phát hiện ảnh bất thường ở cấp độ image-level hay không. Chỉ số image-level phản ánh khả năng xác định một ảnh là bình thường hay bất thường, phù hợp với nhiều kịch bản kiểm tra công nghiệp nơi cần ra quyết định chính là chấp nhận hoặc loại bỏ sản phẩm.
- **Thứ ba**, hệ thống có khả năng định vị vùng bất thường ở cấp độ pixel-level ở mức độ nào. Đây là nhiệm vụ khó hơn vì mô hình không chỉ cần phát hiện ảnh lỗi, mà còn phải xác định vùng lỗi nhỏ trên ảnh.
- **Thứ tư**, Phase 3 có thực hiện được vai trò củng cố offline an toàn hay không. Ở đây, trọng tâm không chỉ là việc checkpoint sau Phase 3 có tăng metric hay không, mà còn là liệu hệ thống có cơ chế từ chối checkpoint nếu cập nhật làm giảm hiệu năng tích lũy hay không.

5.2 Bộ dữ liệu

Thực nghiệm chính được thực hiện trên MVTec AD, một benchmark phổ biến cho phát hiện bất thường công nghiệp [1]. Bộ dữ liệu gồm 15 lớp sản phẩm, bao gồm 5 lớp kết cấu và 10 lớp vật thể. Với mỗi lớp, tập huấn luyện chủ yếu gồm ảnh bình thường, trong khi tập kiểm thử gồm cả ảnh bình thường và ảnh có lỗi. Ngoài nhãn ảnh, MVTec AD còn cung cấp mặt nạ vùng lỗi ở mức điểm ảnh, cho phép đánh giá cả phát hiện ảnh bất thường và định vị vùng lỗi.

Trong thiết lập Continual Learning, mỗi lớp trong MVTec AD được xem như một tác vụ. Các tác vụ được đưa vào hệ thống theo thứ tự cố định:

bottle $\rightarrow$ cable $\rightarrow$ capsule $\rightarrow \dots \rightarrow$ zipper.

Ở mỗi tác vụ, mô hình chỉ cập nhật coreset từ dữ liệu bình thường của tác vụ hiện tại. Khi đánh giá, mô hình được kiểm tra trên dữ liệu test của tác vụ hiện tại và trên tập tích lũy gồm các tác vụ đã học trước đó. Thiết lập này phản ánh bối cảnh công nghiệp trong đó hệ thống lần lượt gặp các loại sản phẩm mới, nhưng vẫn cần duy trì khả năng nhận diện bất thường trên các loại sản phẩm cũ.

Trong quá trình huấn luyện, do tập train của MVTec AD chỉ gồm ảnh bình thường, hệ thống có thể sử dụng cơ chế sinh bất thường giả để kiểm thử tính ổn định của quá trình lọc cập nhật. Tuy nhiên, các mẫu bất thường giả không được đưa vào coreset như mẫu bình thường. Engine huấn luyện chỉ cập nhật coreset bằng các mẫu normal đã được duyệt, nhằm tránh làm ô nhiễm memory bank bởi các mẫu lỗi hoặc nhiễu.

5.3 Cấu hình thực nghiệm

Đường chạy thực nghiệm chính được thực hiện trên toàn bộ 15 nhóm dữ liệu của MVTec AD. Mỗi nhóm dữ liệu được xem như một tác vụ trong chuỗi Continual Learning. backbone được sử dụng là facebook/dinov2-base, ảnh đầu vào được resize về kích thước 224×224 , tạo ra lưới patch 16×16 , tương ứng với 256 patch embedding cho mỗi ảnh. Số lân cận gần nhất khi tính điểm bất thường được đặt bằng $b = 9$. Trong cấu hình báo cáo chính, hệ thống không sử dụng chuẩn hóa pixel score và không áp dụng Gaussian smoothing lên anomaly map.

Các cấu hình chính của đường chạy thực nghiệm gồm:

- Dataset: MVTec AD, 15/15 categories;
- Backbone: facebook/dinov2-base;
- Kích thước ảnh: 224×224 ;
- Patch grid: 16×16 ;
- Số lân cận gần nhất khi scoring: $b = 9$;
- Pixel score normalization: none;

- Gaussian smoothing sigma: 0.0;
- Checkpoint policy: last_only.

Đề án đánh giá hai cấu hình Phase 3. Cấu hình thứ nhất là *Phase 3 conservative*, được dùng làm kết quả báo cáo chính. Cấu hình này ưu tiên tính ổn định, cập nhật backbone rất nhẹ và chỉ chấp nhận checkpoint nếu các chỉ số tích lũy không suy giảm vượt ngưỡng. Cấu hình thứ hai là *Phase 3 max-power*, trong đó NSP2 và CBP được bật, số anchor tăng lên 128, số bước cập nhật là 2, và hai block cuối của backbone được mở khóa. Cấu hình này được dùng như một benchmark thực nghiệm mở rộng nhằm kiểm tra khả năng tích hợp các cơ chế bảo vệ tri thức cũ trong pipeline đầy đủ.

5.4 Chỉ số đánh giá

Nhóm sử dụng bốn nhóm chỉ số chính:

- **Thứ nhất**, image-level AUROC. Chỉ số này đo khả năng phân biệt ảnh bình thường và ảnh bất thường dựa trên điểm bất thường cấp ảnh. AUROC càng cao cho thấy mô hình càng xếp hạng ảnh lỗi cao hơn ảnh bình thường.
- **Thứ hai**, image-level Average Precision. Chỉ số này phản ánh chất lượng xếp hạng trong bối cảnh dữ liệu mất cân bằng, phổ biến trong phát hiện bất thường.
- **Thứ ba**, pixel-level AUROC và pixel-level AUPR. Hai chỉ số này đánh giá chất lượng anomaly map ở mức điểm ảnh. Trong đó, pixel-level AUPR đặc biệt quan trọng vì vùng lỗi thường chiếm tỷ lệ rất nhỏ so với toàn bộ ảnh; do đó bài toán pixel-level có mức mất cân bằng rất lớn.
- **Thứ tư**, cumulative evaluation và Forgetting Measure. Sau khi học qua nhiều tác vụ, mô hình được đánh giá trên tập kiểm thử tích lũy của các tác vụ đã học cho thấy mô hình có duy trì hiệu năng trên các tác vụ cũ hay không. Với ma trận hiệu năng $a_{i,j}$, mức độ quên được tính bằng hiệu năng tốt nhất từng đạt được trên tác vụ j trừ đi hiệu năng cuối cùng trên cùng tác vụ đó, sau đó lấy trung bình trên các tác vụ cũ, tương tự cách đánh giá thường gặp trong học liên tục [10].

Ngoài các chỉ số trên, Phase 3 còn sử dụng metric-gated acceptance. Một checkpoint Phase 3 chỉ được chấp nhận nếu sau cập nhật, final cumulative pixel AUPR không giảm quá ngưỡng -0.005 và final cumulative image AUROC không giảm quá ngưỡng -0.002 . Cơ chế này đảm bảo yêu cầu từ thực tiễn: Một bản cập nhật không nên được đưa vào sử

dụng nếu nó làm suy giảm đáng kể hiệu năng trên dữ liệu lịch sử.

5.5 Kết quả main reportable benchmark trên 15 tác vụ

Bảng dưới đây trình bày kết quả của đường chạy chính trên toàn bộ 15 tác vụ của MVTec AD. Cấu hình được sử dụng là Phase 3 conservative với metric-gated acceptance được bật.

Bảng 2: Kết quả trước và sau Phase 3 conservative trên 15 tác vụ MVTec AD.

Chỉ số	Trước Phase 3	Sau Phase 3	Chênh lệch
Avg image AUROC	0.963984	0.964069	+0.000084
Avg pixel AUROC	0.954502	0.954588	+0.000086
Avg pixel AUPR	0.474363	0.474474	+0.000111
Avg image AP	0.982895	0.982948	+0.000053
Final cumulative image AUROC	0.947035	0.947269	+0.000233
Final cumulative pixel AUPR	0.351101	0.351049	-0.000052

Kết quả cho thấy Phase 3 conservative được metric-gated acceptance chấp nhận trên toàn bộ 15 tác vụ. Final cumulative image AUROC tăng nhẹ từ 0.947035 lên 0.947269, trong khi final cumulative pixel AUPR chỉ giảm rất nhỏ từ 0.351101 xuống 0.351049. Mức suy giảm này là -0.000052 , nhỏ hơn đáng kể so với ngưỡng cho phép -0.005 . Do đó, checkpoint sau Phase 3 được xem là an toàn để chấp nhận theo tiêu chí đánh giá của đồ án.

Mặc dù Phase 3 conservative không tạo ra cải thiện lớn về mặt định lượng, cấu hình này vẫn đạt đúng mục tiêu thiết kế: Thực hiện củng cố offline mà không làm phá vỡ hiệu năng tích lũy. Trong bối cảnh Continual Learning, đây là một yêu cầu quan trọng, vì một checkpoint mới không nên được triển khai chỉ vì cải thiện nhẹ trên một vài chỉ số hiện tại nếu nó gây suy giảm đáng kể trên các tác vụ đã học trước đó.

5.6 Kết quả experimental max-power với NSP2 và CBP

Bên cạnh cấu hình conservative, nhóm cũng thực hiện một cấu hình thực nghiệm mạnh hơn nhằm kiểm tra khả năng tích hợp NSP2 và CBP trong Phase 3. Trong cấu hình này, hệ thống sử dụng 128 anchor, thực hiện 2 bước cập nhật, mở khóa 2 block cuối của backbone, bật LEJEPa surrogate với trọng số 0.1, đặt trọng số patch distillation bằng 5.0, đồng thời bật NSP2 và CBP.

Các thông số chính của Phase 3 max-power gồm:

- top_k_anchors: 128;
- số bước cập nhật: 2;
- số block backbone được mở khóa: 2;
- trọng số LEJEP: 0.1;
- trọng số patch distillation: 5.0;
- NSP2: bật;
- CBP: bật;
- drift: 0.000315;
- patch drift MSE: 0.000672;
- drift threshold: 0.05;
- rollback: không xảy ra.

Bảng 3: Kết quả trước và sau Phase 3 max-power trên 15 tác vụ MVTec AD.

Chỉ số	Trước Phase 3	Sau Phase 3	Chênh lệch
Avg image AUROC	0.963984	0.964111	+0.000127
Avg pixel AUROC	0.954502	0.954800	+0.000298
Avg pixel AUPR	0.474363	0.474530	+0.000167
Avg image AP	0.982895	0.982999	+0.000104
Final cumulative image AUROC	0.947034	0.947791	+0.000757
Final cumulative pixel AUPR	0.351101	0.350401	-0.000701

Cấu hình max-power cũng được metric-gated acceptance chấp nhận. Final cumulative image AUROC tăng từ 0.947034 lên 0.947791, tương ứng mức tăng +0.000757. Final cumulative pixel AUPR giảm từ 0.351101 xuống 0.350401, tương ứng mức giảm -0.000701. Mức giảm này vẫn nằm trong ngưỡng an toàn -0.005, do đó checkpoint sau Phase 3 max-power được xem là hợp lệ theo tiêu chí metric-gated acceptance.

So với cấu hình conservative, max-power tạo ra mức tăng rõ hơn ở final cumulative image AUROC và các chỉ số trung bình, đặc biệt là avg pixel AUROC và avg pixel AUPR. Tuy nhiên, nó cũng làm final cumulative pixel AUPR giảm nhiều hơn so với cấu hình

conservative. Điều này cho thấy khi tăng cường mức độ cập nhật backbone, hệ thống có thể cải thiện biểu diễn ở cấp độ ảnh và điểm trung bình, nhưng vẫn cần kiểm soát chặt để tránh làm suy giảm khả năng định vị pixel tích lũy. Kết quả này củng cố vai trò của metric-gated acceptance: Thay vì giả định mọi cập nhật mạnh đều tốt, hệ thống đánh giá trực tiếp tác động của checkpoint mới trước khi chấp nhận nó.

5.7 Kiểm thử cơ chế

Ngoài các benchmark chính, nhóm thực hiện một bộ smoke test nhằm kiểm tra tính đúng đắn của các module cốt lõi trong pipeline. Kết quả cho thấy toàn bộ 72 kiểm thử đều vượt qua. Các cơ chế được kiểm tra bao gồm đường chạy TITANS memory, cập nhật và scoring của CADIC coreset, NSP2 projection, Subspace Recycling và CBP reset path.

Bảng 4: Kết quả kiểm thử các cơ chế chính trong pipeline.

Cơ chế	Kết quả
TITANS memory path	Pass
CADIC coreset update/scoring	Pass
NSP2 projection path	Pass
Subspace Recycling path	Pass
CBP reset path	Pass
Tổng số smoke tests	72/72 pass

Kết quả này không được dùng để khẳng định cải thiện định lượng của từng cơ chế riêng lẻ mà để xác nhận rằng các thành phần chính của hệ thống đã được tích hợp và có thể chạy trong cùng một pipeline. Theo đó, việc NSP2, Subspace Recycling và CBP đều có đường chạy kiểm thử giúp đảm bảo rằng các module mở rộng của Phase 3 tối thiểu là đã work được với nhau và tích hợp vào hệ thống.

5.8 Thảo luận kết quả thực nghiệm

Các kết quả thực nghiệm cho thấy pipeline đề xuất đạt được mục tiêu chính của đề án: Xây dựng một hệ thống CAD có thể chạy qua toàn bộ 15 tác vụ của MVtec AD, thực hiện Phase 3 consolidation và sử dụng metric-gated acceptance để kiểm soát checkpoint sau cập nhật.

Ở cấp độ ảnh, hệ thống đạt hiệu năng tương đối mạnh. Trong cấu hình conservative, avg image AUROC sau Phase 3 đạt 0.964069, còn final cumulative image AUROC đạt

0.947269. Trong cấu hình max-power, final cumulative image AUROC tăng lên 0.947791. Điều này cho thấy đặc trưng từ DINOv2 kết hợp với CADIC coreset và patch nearest-neighbor scoring vẫn giữ được khả năng phân biệt ảnh bất thường và ảnh bình thường qua chuỗi tác vụ liên tục.

Ở cấp độ pixel, avg pixel AUPR sau Phase 3 đạt 0.474474 trong cấu hình conservative và 0.474530 trong cấu hình max-power. Tuy nhiên, final cumulative pixel AUPR chỉ đạt 0.351049 ở cấu hình conservative và 0.350401 ở cấu hình max-power. Khoảng cách giữa avg pixel AUPR và final cumulative pixel AUPR cho thấy định vị bất thường ở mức điểm ảnh khi đánh giá tích lũy vẫn là thách thức lớn nhất của hệ thống.

Kết quả này phù hợp với bản chất của định hướng thiết kế ban đầu của nhóm. Hệ thống ưu tiên một pipeline nhẹ dựa trên coreset embedding, patch nearest-neighbor scoring và utility-guided anchor replay, thay vì sử dụng generative replay để tái tạo dữ liệu lịch sử với chi tiết pixel cao. Cách tiếp cận này giúp pipeline nhẹ, dễ kiểm soát hơn, ít phụ thuộc vào mô hình sinh hơn và phù hợp hơn với thiết lập edge-cloud. Tuy nhiên, nó cũng làm hệ thống gặp bất lợi ở bài toán cumulative pixel localization, nơi các chi tiết lỗi nhỏ của tác vụ cũ cần được bảo toàn tốt hơn.

Một điểm quan trọng là cả hai cấu hình Phase 3 đều được metric-gated acceptance chấp nhận. Cấu hình conservative gần như không làm suy giảm final cumulative pixel AUPR, trong khi cấu hình max-power cải thiện final cumulative image AUROC nhiều hơn nhưng làm giảm pixel AUPR tích lũy rõ hơn, dù vẫn nằm trong ngưỡng cho phép. Điều này cho thấy hệ thống đã có cơ chế kiểm soát cập nhật hợp lý: Các checkpoint sau Phase 3 không được chấp nhận dựa trên loss hoặc drift đơn thuần, mà phải vượt qua đánh giá before/after trên các chỉ số tích lũy.

Chương 6

Đánh giá và thảo luận

6.1 Đánh giá kiến trúc đề xuất

Kiến trúc đề xuất thể hiện cách tiếp cận học liên tục theo hướng phân tầng. Thay vì giải quyết toàn bộ bài toán bằng một cơ chế duy nhất, hệ thống chia trách nhiệm cho nhiều thành phần: backbone trích xuất đặc trưng, TITANS-style memory thích nghi nhanh, ACC gating kiểm soát cập nhật, CADIC coreset lưu trữ thức ổn định, utility-guided anchor replay cung cấp một lượng nhỏ ví dụ đại diện cho Phase 3 và metric-gated acceptance bảo vệ quá trình triển khai checkpoint mới.

Cách tổ chức này có ưu điểm là rõ ràng về mặt vai trò. Phase 1-2 phù hợp với thiết lập online hoặc edge, nơi hệ thống cần xử lý dữ liệu tuần tự và cập nhật bộ nhớ mà không tinh chỉnh toàn bộ backbone. Phase 3 phù hợp với thiết lập cloud hoặc offline, nơi có thể thực hiện củng cố chậm hơn, đo before/after metrics và quyết định có chấp nhận checkpoint mới hay không. Đây là một điểm khác biệt so với các pipeline tĩnh, trong đó mô hình thường được huấn luyện một lần rồi sử dụng mà không có cơ chế cập nhật rõ ràng.

Kiến trúc này cũng phù hợp với yêu cầu công nghiệp ở mức nhất định. Trong nhiều dây chuyền sản xuất, việc phát hiện ảnh lỗi ở cấp độ image-level là yêu cầu quan trọng đầu tiên. Kết quả image AUROC cao cho thấy hướng tiếp cận memory-based trên DINOv2 có tính khả dụng trong kịch bản này. Đồng thời, việc không phụ thuộc vào generative replay trong online path giúp pipeline dễ kiểm soát hơn so với các phương pháp cần sinh lại dữ liệu lịch sử bằng Diffusion Models.

Kết quả full 15-task cho thấy kiến trúc đề xuất đã vượt qua mức kiểm chứng ban đầu. Cả cấu hình Phase 3 conservative và cấu hình Phase 3 max-power đều hoàn thành before/after evaluation trên toàn bộ 15 tác vụ MVTec AD và đều được metric-gated acceptance chấp nhận. Điều này cho thấy pipeline không chỉ chạy được ở mức smoke test hoặc một số tác vụ đầu mà đã có khả năng vận hành trên toàn bộ chuỗi tác vụ chính của benchmark.

Tuy nhiên, hai cấu hình cũng thể hiện rõ sự đánh đổi giữa mức độ cập nhật và độ an toàn. Cấu hình conservative gần như giữ nguyên final cumulative pixel AUPR, trong khi cấu hình max-power cải thiện image-level cumulative AUROC nhiều hơn nhưng làm

pixel cumulative AUPR giảm mạnh hơn. Mặc dù cả hai mức giảm đều nằm trong ngưỡng metric-gated acceptance, kết quả này cho thấy pixel-level cumulative localization vẫn nhạy cảm với các cập nhật backbone và cần được bảo vệ chặt chẽ.

6.2 Ba cơ chế bảo vệ tri thức cũ

Trong pipeline đề xuất, có thể phân biệt ba cơ chế chính nhằm bảo vệ tri thức cũ.

- **Cơ chế thứ nhất – CADIC coreset:** coreset đóng vai trò stable memory, lưu các biểu diễn bình thường đại diện qua nhiều tác vụ. Khi mô hình kiểm thử trên ảnh mới, patch embedding của ảnh được so sánh với patch bank trong coreset. Nhờ đó, hệ thống không cần giữ toàn bộ dữ liệu lịch sử nhưng vẫn có một bộ nhớ đại diện cho phân phối bình thường của các tác vụ đã học.
- **Cơ chế thứ hai – utility-guided anchor replay:** Đây là dạng replay nhẹ, khác với generative replay. Hệ thống chọn một số anchor có utility score cao từ coreset và cân bằng theo tác vụ để sử dụng trong Phase 3. Các anchor này giúp mô hình có cơ hội “xem lại” một số ví dụ quan trọng của các tác vụ cũ khi củng cố backbone. Cơ chế này đặc biệt cần thiết cho pixel-level localization, vì chỉ lưu embedding tổng quát là không đủ để duy trì chi tiết cục bộ vững vàng.
- **Cơ chế thứ ba – NSP2 kết hợp với CBP:** Trong Phase 3 max-power, NSP2 định hướng cách cập nhật sao cho ít phá vỡ tri thức cũ hơn, còn CBP hỗ trợ theo dõi và tái khởi tạo các đơn vị có utility score thấp. Hai cơ chế này được dùng để kiểm tra khả năng mở rộng của pipeline khi mức cập nhật backbone mạnh hơn cấu hình conservative.

Bên ngoài ba cơ chế này, metric-gated acceptance đóng vai trò lớp bảo vệ triển khai. Nếu một checkpoint sau Phase 3 làm giảm cumulative pixel AUPR hoặc image AUROC vượt ngưỡng cho phép, checkpoint đó bị từ chối. Đây là một cơ chế quan trọng trong học liên tục vì nó chuyển câu hỏi từ “mô hình có cập nhật được không” sang “bản cập nhật có an toàn để triển khai không”.

Trong kết quả thực nghiệm hiện tại, CADIC coreset và utility-guided anchor replay là hai cơ chế trực tiếp tham gia vào đường chạy báo cáo chính. NSP2 và CBP được bật trong cấu hình max-power và đã hoàn thành full 15-task before/after evaluation với quyết định accepted. Tuy nhiên, mức cải thiện định lượng của NSP2/CBP chưa đủ lớn để kết luận rằng chúng là nguyên nhân chính tạo ra tăng trưởng metric. Do đó, trong phạm vi đề án,

NSP2 và CBP được xem là các cơ chế mở rộng đã được tích hợp thành công và kiểm chứng đường chạy nhưng vẫn cần thêm ablation riêng để đánh giá đóng góp độc lập của từng thành phần.

6.3 So sánh định tính với hướng replay-based CAD

Replay-based continual learning là một hướng mạnh và phổ biến để chống catastrophic forgetting. Trong bài toán CAD, ReplayCAD là một đại diện tiêu biểu khi sử dụng diffusion-based generative replay để phát lại dữ liệu lịch sử có chi tiết pixel [16]. Ưu điểm của hướng này là mô hình có thể tiếp xúc lại với dữ liệu cũ ở dạng gần với ảnh thật, nhờ đó bảo toàn tốt hơn các đặc trưng cục bộ cần cho segmentation.

So với hướng đó, phương pháp trong đề án không cố gắng sinh lại toàn bộ dữ liệu lịch sử. Thay vào đó, hệ thống chỉ lưu một lượng nhỏ anchor đại diện, được chọn bằng utility score. Cách tiếp cận này làm giảm độ phức tạp của pipeline và hạn chế phụ thuộc vào mô hình sinh, nhưng cũng dẫn đến đánh đổi rõ ràng: Image-level detection vẫn mạnh, current pixel localization ở mức chấp nhận được, nhưng cumulative pixel localization thấp hơn.

Sự đánh đổi này có thể được nhìn nhận tùy theo góc độ triển khai. Nếu mục tiêu của hệ thống là segmentation mask có độ chính xác cao nhất và có thể chấp nhận pipeline offline phức tạp, generative replay là một lựa chọn mạnh. Ngược lại, nếu mục tiêu là một hệ thống edge-cloud có khả năng phát hiện sản phẩm lỗi/không lỗi và không nhất thiết yêu cầu phân đoạn vị trí lỗi với độ chính xác cao nhất, thêm vào đó là các tính năng như cập nhật tuần tự, lưu trữ có kiểm soát, dễ audit và có cơ chế chấp nhận checkpoint rõ ràng, hướng Nested Learning với utility-guided anchor replay là một lựa chọn hợp lý hơn.

Do đó, phương pháp đề xuất không nên được định vị như một thay thế tuyệt đối cho ReplayCAD. Thay vào đó, nó đại diện cho một nhánh khác của CAD: Nhánh nhẹ hơn, tính module hóa cao hơn, tập trung vào online memory update và safe cloud consolidation. Điểm yếu về cumulative pixel localization được ghi nhận như một giới hạn cần cải thiện trong các bước nghiên cứu tiếp theo.

6.4 Hạn chế

Hạn chế đầu tiên và quan trọng nhất của đề án là cumulative pixel localization vẫn còn thấp. Mặc dù avg pixel AUPR sau Phase 3 đạt khoảng 0.4745, final cumulative pixel AUPR chỉ đạt khoảng 0.3510. Điều này cho thấy hệ thống vẫn còn khó khăn trong việc

duy trì khả năng định vị bất thường ở mức điểm ảnh khi đánh giá trên toàn bộ chuỗi tác vụ.

Hạn chế thứ hai là Phase 3 hiện chủ yếu chứng minh tính an toàn hơn là cải thiện mạnh về metric. Cả cấu hình conservative và max-power đều được metric-gated acceptance chấp nhận, nhưng mức tăng ở các chỉ số trung bình còn nhỏ. Max-power cải thiện final cumulative image AUROC tốt hơn conservative, nhưng đổi lại làm final cumulative pixel AUPR giảm nhiều hơn. Do đó, Phase 3 hiện nên được xem là một cơ chế safe consolidation thay vì một cơ chế tối ưu hóa hiệu năng triệt để.

Hạn chế thứ ba là tác động riêng lẻ của NSP2 và CBP chưa được tách bằng ablation đầy đủ. Cấu hình max-power bật cả NSP2 và CBP, đồng thời thay đổi nhiều siêu tham số như số anchor, số block mở khóa và số bước cập nhật. Vì vậy, kết quả accepted của max-power chứng minh pipeline tích hợp hoạt động ổn định, nhưng chưa đủ để quy kết mức thay đổi metric cho riêng NSP2 hoặc CBP.

Hạn chế thứ tư là pipeline đánh giá còn tốn thời gian khi chạy full 15-task với before/after evaluation và experimental branch. Đây là chi phí chấp nhận được trong bối cảnh benchmark nghiên cứu, nhưng khi triển khai thực tế cần tách riêng đường suy luận production khỏi đường đánh giá đầy đủ.

6.5 Hướng phát triển

Từ các kết quả thực nghiệm trên MVTEC AD, có thể thấy pipeline Meta-NATH CAD đã đạt được mục tiêu chính ở mức prototype nghiên cứu: Hệ thống có thể vận hành qua chuỗi tác vụ liên tục, duy trì bộ nhớ normal bằng CADIC coreset, thực hiện củng cố offline ở Phase 3 và sử dụng cơ chế metric-gated acceptance để kiểm soát checkpoint sau cập nhật. Tuy nhiên, kết quả cũng cho thấy một hạn chế rõ ràng: Mặc dù image-level AUROC duy trì ở mức tương đối tốt, cumulative pixel AUPR vẫn là điểm yếu chính của hệ thống. Điều này cho thấy hướng phát triển tiếp theo không nên chỉ tập trung vào việc làm backbone cập nhật mạnh hơn, mà cần ưu tiên bảo toàn cấu trúc patch-level và ổn định phân phối điểm bất thường qua nhiều tác vụ.

Trong ngắn hạn, hướng ưu tiên đầu tiên là cải thiện khả năng định vị bất thường ở cấp độ pixel. Một hướng khả thi là hiệu chỉnh score theo task hoặc theo class để giảm sai lệch phân phối giữa các nhóm sản phẩm. Hiện tại, patch nearest-neighbor scoring sử dụng cùng một cơ chế tính điểm cho toàn bộ chuỗi tác vụ, trong khi mỗi category trong MVTEC

AD có đặc điểm hình học, texture và tỷ lệ lỗi khác nhau. Do đó, việc bổ sung task-aware hoặc class-aware calibration có thể giúp cumulative pixel AUPR ổn định hơn khi đánh giá trên các tác vụ đã học trước đó.

Hướng thứ hai là cải thiện cơ chế utility-guided anchor replay. Trong thiết kế hiện tại, anchor được chọn từ coreset nhằm đại diện cho các vùng normal quan trọng khi thực hiện Phase 3 consolidation. Tuy nhiên, utility score có thể được mở rộng để phản ánh nhiều yếu tố hơn, chẳng hạn mức độ đại diện của mẫu, độ gần biên phân phối normal, độ hiếm của patch embedding và ảnh hưởng của mẫu đến Forgetting Measure. Cách tiếp cận này vẫn giữ được ưu điểm của hệ thống là không phụ thuộc vào diffusion-based generative replay, nhưng tăng khả năng bảo toàn chi tiết của các tác vụ cũ.

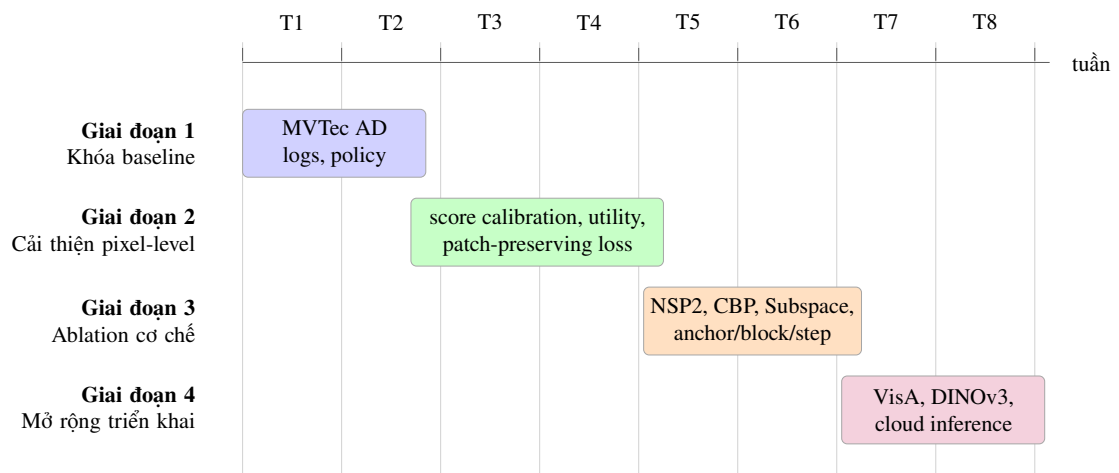
Hướng thứ ba là thiết kế objective cho Phase 3 theo hướng bảo toàn không gian patch-level. Các kết quả hiện tại cho thấy cập nhật backbone có thể cải thiện nhẹ image-level metric nhưng vẫn có nguy cơ làm giảm cumulative pixel AUPR. Vì vậy, Phase 3 không chỉ nên dựa trên distillation ở mức embedding tổng quát mà cần có thêm ràng buộc bảo toàn lân cận patch, cấu trúc anomaly map hoặc phân phối score trước và sau consolidation. Tóm lại, mục tiêu của Phase 3 trong các phiên bản tiếp theo không chỉ là cập nhật an toàn ở cấp độ ảnh mà còn phải giữ ổn định khả năng định vị lỗi ở cấp độ pixel.

Trong trung hạn, cần thực hiện ablation study cho từng thành phần mở rộng của hệ thống. Cấu hình hiện tại đã tích hợp được NSP2, CBP và Subspace Recycling ở mức đường chạy thực nghiệm, nhưng chưa đủ để kết luận riêng rẽ đóng góp của từng cơ chế. Do đó, các thí nghiệm tiếp theo nên tách riêng từng module: Chạy Phase 3 không NSP2/CBP, bật riêng NSP2, bật riêng CBP, sau đó so sánh với cấu hình bật đồng thời. Việc này giúp xác định cơ chế nào thật sự đóng góp vào stability, cơ chế nào chủ yếu đóng vai trò regularization và cơ chế nào có thể gây tác dụng phụ đối với pixel-level localization.

Trong dài hạn, pipeline có thể được mở rộng theo hai hướng. Hướng thứ nhất là đánh giá trên benchmark khác như VisA để kiểm tra khả năng tổng quát hóa ngoài MVTec AD. Hướng thứ hai là thử nghiệm backbone mới hơn, chẳng hạn DINOv3 hoặc các Vision Foundation Models có biểu diễn patch-level mạnh hơn. Tuy nhiên, các mở rộng này chỉ nên được thực hiện sau khi pipeline DINOv2 hiện tại đã được locking ổn định, vì thay đổi backbone quá sớm có thể làm khó khăn cho việc phân tích nguyên nhân cải thiện hoặc suy giảm metric.

Hình 7 trình bày lộ trình phát triển đề xuất cho hệ thống. Lộ trình này được chia thành

bốn giai đoạn chính: củng cố kết quả hiện tại, cải thiện pixel-level localization, đánh giá độc lập các cơ chế mở rộng, và mở rộng benchmark/triển khai. Các công việc cải thiện cumulative pixel AUPR và ablation study được ưu tiên trước khi mở rộng sang benchmark hoặc backbone mới, vì đây là hai nhóm công việc trực tiếp giải quyết hạn chế quan trọng nhất của kết quả hiện tại.



Hình 7: Lộ trình phát triển đề xuất cho Meta-NATH CAD. Các bước ngắn hạn tập trung vào việc khóa baseline và cải thiện cumulative pixel localization; các bước trung hạn đánh giá độc lập NSP2, CBP và Subspace Recycling; các bước dài hạn mở rộng sang VisA, DINOv3 và tối ưu triển khai cloud/industrial inference.

Chương 7

Tổng kết

Đồ án đã nghiên cứu bài toán CAD trên ảnh công nghiệp theo định hướng Nested Learning. Xuất phát từ hạn chế của các hệ thống phát hiện bất thường tĩnh, đề tài tập trung vào bối cảnh dữ liệu được đưa đến theo chuỗi tác vụ, trong đó mô hình phải tiếp thu nhóm dữ liệu mới mà vẫn duy trì hiệu năng trên các nhóm dữ liệu đã học. Đây là một bài toán có ý nghĩa thực tế trong công nghiệp, nơi điều kiện vận hành, loại sản phẩm và dạng lỗi có thể thay đổi theo thời gian.

Về mặt phương pháp, đồ án đã xây dựng một pipeline gồm nhiều tầng học. backbone DINOv2 được sử dụng để trích xuất đặc trưng ảnh ở cấp độ toàn cục và cấp độ patch. TITANS-style Fast Memory hỗ trợ thích nghi nhanh với dữ liệu đầu vào. ACC Gating kiểm soát việc cập nhật mẫu vào bộ nhớ ổn định. CADIC coreset lưu giữ các biểu diễn bình thường đại diện qua nhiều tác vụ. Utility-guided anchor replay chọn một số mẫu đại diện có giá trị cao để sử dụng trong Phase 3. Cuối cùng, Phase 3 thực hiện củng cố offline bằng patch-token distillation, refresh coreset và metric-gated acceptance.

Về mặt thực nghiệm, hệ thống đã được đánh giá trên toàn bộ 15 tác vụ của MVTec AD. Ở cấu hình báo cáo chính Phase 3 conservative, checkpoint sau Phase 3 được metric-gated acceptance chấp nhận. Final cumulative image AUROC tăng từ 0.947035 lên 0.947269, trong khi final cumulative pixel AUPR chỉ giảm rất nhỏ từ 0.351101 xuống 0.351049, nằm xa trong ngưỡng chấp nhận. Điều này cho thấy cơ chế củng cố offline có thể vận hành trên toàn bộ chuỗi tác vụ mà không gây suy giảm đáng kể hiệu năng tích lũy.

Đồ án cũng kiểm thử một cấu hình experimental max-power, trong đó NSP2 và CBP được bật, số anchor tăng lên 128, số block backbone được cập nhật tăng lên 2, và LEJEPa surrogate được sử dụng. Cấu hình này cũng được metric-gated acceptance chấp nhận. Final cumulative image AUROC tăng từ 0.947034 lên 0.947791, trong khi final cumulative pixel AUPR giảm từ 0.351101 xuống 0.350401, vẫn nằm trong ngưỡng an toàn. Kết quả này cho thấy các cơ chế mở rộng như NSP2 và CBP có thể được tích hợp vào pipeline đầy đủ mà không làm phá vỡ hiệu năng tích lũy, dù vẫn cần thêm ablation để kết luận đóng góp riêng của từng cơ chế.

Ngoài benchmark chính, nhóm cũng thực hiện kiểm thử cơ chế với tổng cộng 72

smoke tests. Các đường chạy TITANS memory, CADIC coreset update/scoring, NSP2 projection, Subspace Recycling và CBP reset đều vượt qua kiểm thử. Điều này xác nhận rằng các module lõi của hệ thống không chỉ được mô tả ở mức thiết kế, mà đã được hiện thực và có thể phối hợp trong cùng một codebase.

Từ các kết quả trên, có thể kết luận rằng đồ án đã hiện thực được một prototype Nested Learning cho CAD với đầy đủ các thành phần chính: Vision Backbone, Fast Memory, gating, coreset, anchor replay nhẹ, consolidation offline và metric-gated acceptance.

Bên cạnh các kết quả đạt được, đồ án cũng chỉ ra giới hạn quan trọng của hướng tiếp cận. Cumulative pixel localization vẫn còn thấp so với avg pixel AUPR, cho thấy việc bảo toàn chi tiết pixel của các tác vụ cũ vẫn là thách thức lớn. Nguyên nhân chính đến từ việc hệ thống ưu tiên coreset embedding và utility-guided anchor replay nhẹ thay vì generative replay có độ trung thực cao. Đây là một đánh đổi có chủ đích giữa tính nhẹ, khả năng kiểm soát và chất lượng định vị pixel tích lũy.

Trong tương lai, hướng nghiên cứu nên tập trung vào hiệu chỉnh score theo task hoặc class, cải thiện utility score cho anchor replay, thiết kế objective bảo toàn pixel-level neighborhood trong Phase 3 và đánh giá sâu hơn vai trò riêng lẻ của NSP2, CBP cũng như Subspace Recycling. Ngoài ra, pipeline hiện tại có thể được mở rộng sang DINOv3 hoặc các Vision Foundation Models mới hơn sau khi baseline DINOv2 đã được khóa ổn định.

Tóm lại, đề tài đã đạt được mục tiêu xây dựng và đánh giá một hệ thống CAD theo hướng Nested Learning. Hệ thống thể hiện hiệu quả tốt ở cấp độ phát hiện ảnh bất thường, có cơ chế bộ nhớ và củng cố phù hợp với thiết lập học tuần tự, đồng thời cung cấp phân tích rõ ràng về đánh đổi giữa pipeline nhẹ, khả năng cập nhật an toàn và khả năng bảo toàn chi tiết pixel trong môi trường công nghiệp.

Tài liệu

- [1] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, “MVTec AD - A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9592–9600. [Online]. Available: https://openaccess.thecvf.com/content_CVPR_2019/papers/Bergmann_MVTec_AD_-_A_Comprehensive_Real-World_Dataset_for_Unsupervised_Anomaly_CVPR_2019_paper.pdf
- [2] T. Defard, A. Setkov, A. Loesch, and R. Audigier, “PaDiM: A patch distribution modeling framework for anomaly detection and localization,” in *Pattern Recognition. ICPR International Workshops and Challenges*, 2021, pp. 475–489. [Online]. Available: <https://arxiv.org/abs/2011.08785>
- [3] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, “Towards total recall in industrial anomaly detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 14 318–14 328. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2022/papers/Roth_Towards_Total_Recall_in_Industrial_Anomaly_Detection_CVPR_2022_paper.pdf
- [4] L. Wang, X. Zhang, H. Su, and J. Zhu, “A comprehensive survey of continual learning: Theory, method and application,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 8, pp. 5362–5383, 2024. [Online]. Available: <https://doi.org/10.1109/TPAMI.2024.3367329>
- [5] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly Detection: A Survey,” *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009. [Online]. Available: <https://doi.org/10.1145/1541880.1541882>
- [6] Z. Li *et al.*, “A survey of deep learning for industrial visual anomaly detection,” *Artificial Intelligence Review*, 2025. [Online]. Available: <https://link.springer.com/article/10.1007/s10462-025-11287-7>
- [7] D. Gudovskiy, S. Ishizaka, and K. Kozuka, “CFLOW-AD: Real-time unsupervised anomaly detection with localization via conditional normalizing flows,” in *Proceedings of the IEEE/CVF Winter Conference*

- on Applications of Computer Vision (WACV)*, 2022, pp. 715–724. [Online]. Available: https://openaccess.thecvf.com/content/WACV2022/papers/Gudovskiy_CFLOW-AD_Real-Time_Unsupervised_Anomaly_Detection_With_Localization_via_Conditional_Normalizing_WACV_2022_paper.pdf
- [8] W. Li, J. Zhan, J. Wang, B. Xia, B.-B. Gao, J. Liu, C. Wang, and F. Zheng, “Towards continual adaptation in industrial anomaly detection,” in *Proceedings of the 30th ACM International Conference on Multimedia*, 2022, pp. 2871–2880. [Online]. Available: <https://doi.org/10.1145/3503161.3548232>
- [9] N. Bugarin, J. Bugaric, M. Barusco, D. Dalle Pezze, and G. A. Susto, “Unveiling the anomalies in an ever-changing world: A benchmark for pixel-level anomaly detection in continual learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2024, pp. 4065–4074. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2024W/CLVISION/html/Bugarin_Unveiling_the_Anomalies_in_an_Ever-Changing_World_A_Benchmark_for_CVPRW_2024_paper.html
- [10] G. I. Parisi, R. Kemker, J. L. Part, C. Kanan, and S. Wermter, “Continual lifelong learning with neural networks: A review,” *Neural Networks*, vol. 113, pp. 54–71, 2019. [Online]. Available: <https://doi.org/10.1016/j.neunet.2019.01.012>
- [11] M. McCloskey and N. J. Cohen, “Catastrophic interference in connectionist networks: The sequential learning problem,” in *Psychology of Learning and Motivation*, vol. 24. Academic Press, 1989, pp. 109–165.
- [12] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. S. Torr, “Riemannian walk for incremental learning: Understanding forgetting and intransigence,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 532–547. [Online]. Available: https://openaccess.thecvf.com/content_ECCV_2018/html/Arslan_Chaudhry_Riemannian_Walk_ECCV_2018_paper.html
- [13] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska *et al.*, “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017. [Online]. Available: <https://doi.org/10.1073/pnas.1611835114>

- [14] D. Dalle Pezze, E. Anello, C. Masiero, and G. A. Susto, “Continual learning approaches for anomaly detection,” *Evolving Systems*, vol. 16, no. 111, 2025. [Online]. Available: <https://doi.org/10.1007/s12530-025-09732-7>
- [15] G. Yang, Z. Deng, and J. Man, “CADIC: Continual anomaly detection based on incremental coreset,” arXiv preprint arXiv:2511.08634, 2025. [Online]. Available: <https://arxiv.org/abs/2511.08634>
- [16] L. Hu, Z. Gan, L. Deng, J. Liang, L. Liang, S. Huang, and T. Chen, “ReplayCAD: Generative diffusion replay for continual anomaly detection,” arXiv preprint arXiv:2505.06603, 2025. [Online]. Available: <https://arxiv.org/abs/2505.06603>
- [17] J. Liu, K. Wu, Q. Nie, Y. Chen, B.-B. Gao, Y. Liu, J. Wang, C. Wang, and F. Zheng, “Unsupervised Continual Anomaly Detection with Contrastively-Learned Prompt,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 4, 2024, pp. 3639–3647. [Online]. Available: <https://doi.org/10.1609/aaai.v38i4.28153>
- [18] A. Behrouz, M. Razaviyayn, P. Zhong, and V. Mirrokni, “Nested Learning: The illusion of deep learning architectures,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, arXiv version: <https://arxiv.org/abs/2512.24695>. [Online]. Available: <https://neurips.cc/virtual/2025/loc/san-diego/poster/116123>
- [19] Y. Sun, X. Li, K. Dalal, J. Xu, A. Vikram, G. Zhang, Y. Dubois, X. Chen, X. Wang, S. Koyejo, T. Hashimoto, and C. Guestrin, “Learning to (learn at test time): Rnns with expressive hidden states,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.04620>
- [20] A. Behrouz, P. Zhong, and V. Mirrokni, “Titans: Learning to memorize at test time,” arXiv preprint arXiv:2501.00663, 2025. [Online]. Available: <https://arxiv.org/abs/2501.00663>
- [21] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, “DINOv2: Learning robust visual features without supervision,” arXiv preprint arXiv:2304.07193, 2023, <https://arxiv.org/abs/2304.07193>.

- [22] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, “Self-supervised learning from images with a joint-embedding predictive architecture,” *arXiv preprint arXiv:2301.08243*, 2023. [Online]. Available: <https://arxiv.org/abs/2301.08243>
- [23] Y. Lu, S. Zhang, D. Cheng, Y. Xing, N. Wang, P. Wang, and Y. Zhang, “Visual prompt tuning in null space for continual learning,” *arXiv preprint arXiv:2406.05658*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.05658>
- [24] S. Dohare, R. S. Sutton, and A. R. Mahmood, “Continual backprop: Stochastic gradient descent with persistent randomness,” *arXiv preprint arXiv:2108.06325*, 2021. [Online]. Available: <https://arxiv.org/abs/2108.06325>